
Git & GitHub

박성훈

E noii@kkokiyo.app

H <https://noii.kr>

現 꼭코퍼레이션 대표이사

現 에이아이제로엑스 대표이사

前 셀러비코리아 대표이사

前 건국대학교 강의 교수

웹프로그래밍

컴퓨터프로그래밍2

컴퓨팅적사고

건국대학교

컴퓨터/정보통신공학 박사 수료

컴퓨터/정보통신공학 석사 졸업

컴퓨터공학 학사 졸업



SESSION 1

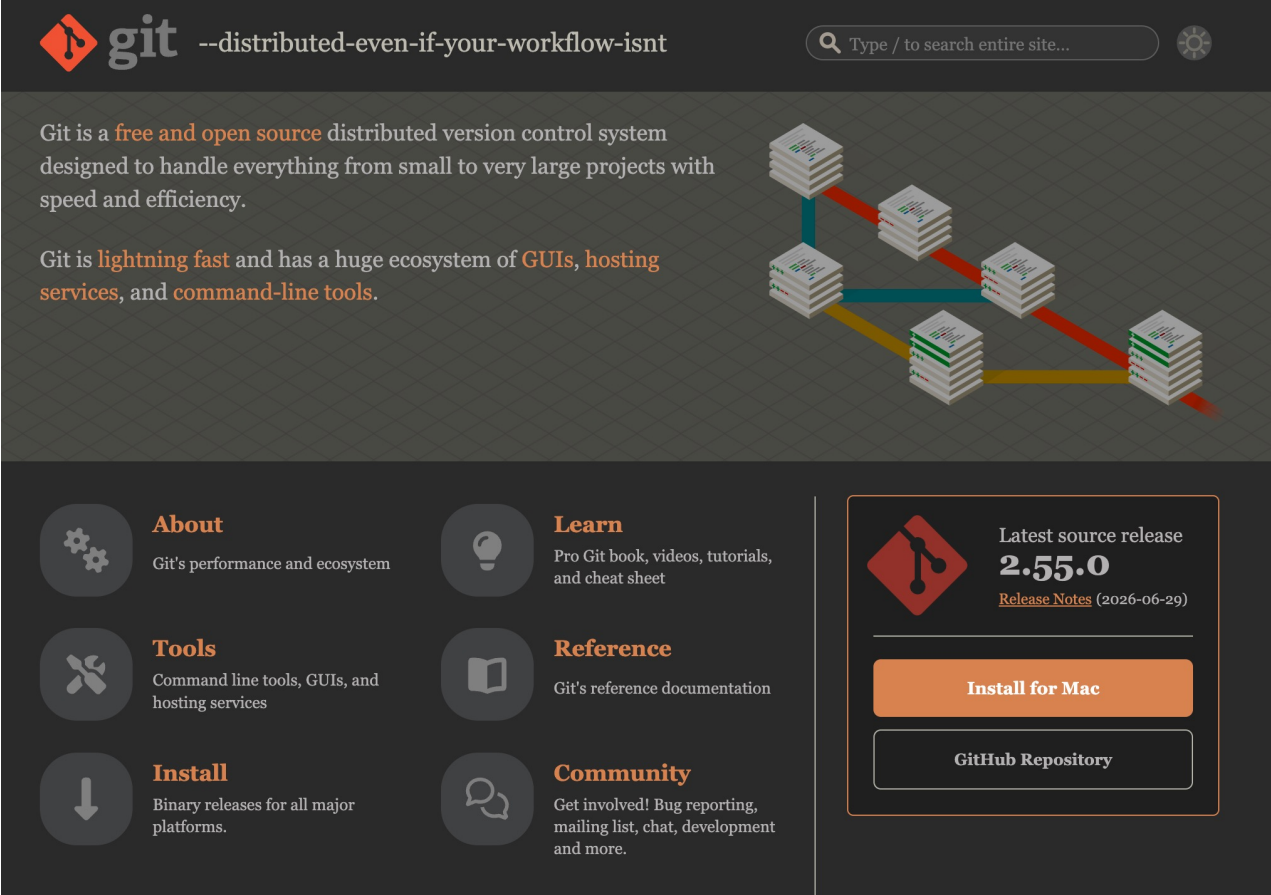
환경 점검

INSTALL

Git 설치하기

홈페이지를 방문해서 Git을 설치하세요

<https://git-scm.com/>



The screenshot shows the Git website homepage. At the top, the Git logo is followed by the tagline "--distributed-even-if-your-workflow-isnt". A search bar is on the right. The main content area features a description of Git as a "free and open source" distributed version control system, followed by a diagram of a distributed version control system with multiple repositories connected by lines. Below this, there are six icons with corresponding text: "About" (performance and ecosystem), "Tools" (command line tools, GUIs, and hosting services), "Install" (binary releases for all major platforms), "Learn" (Pro Git book, videos, tutorials, and cheat sheet), "Reference" (Git's reference documentation), and "Community" (Get involved! Bug reporting, mailing list, chat, development and more). On the right side, there is a section for the "Latest source release 2.55.0" with a link to "Release Notes (2026-06-29)" and two buttons: "Install for Mac" and "GitHub Repository".

git --distributed-even-if-your-workflow-isnt

Type / to search entire site...

Git is a **free and open source** distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

Git is **lightning fast** and has a huge ecosystem of **GUIs**, **hosting services**, and **command-line tools**.

About
Git's performance and ecosystem

Tools
Command line tools, GUIs, and hosting services

Install
Binary releases for all major platforms.

Learn
Pro Git book, videos, tutorials, and cheat sheet

Reference
Git's reference documentation

Community
Get involved! Bug reporting, mailing list, chat, development and more.

Latest source release
2.55.0
[Release Notes](#) (2026-06-29)

Install for Mac

GitHub Repository

Git 설치하기

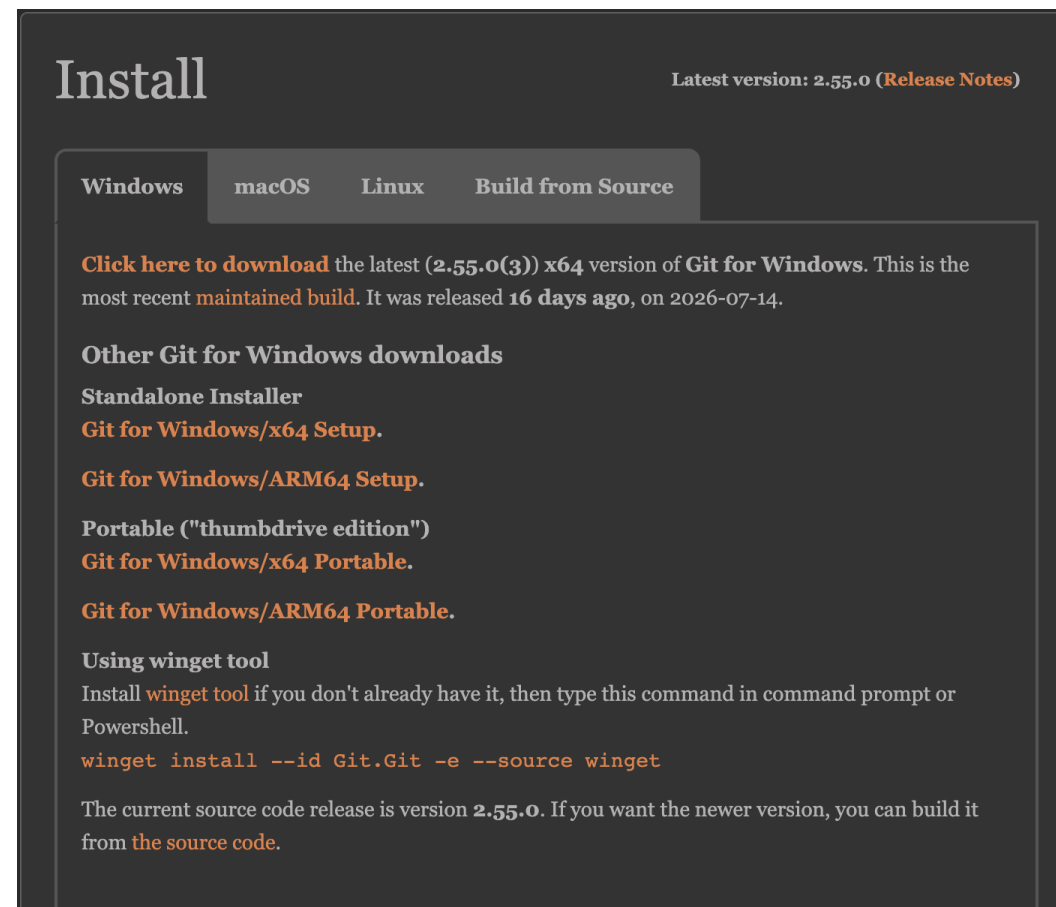
홈페이지를 방문해서 Git을 설치하세요

<https://git-scm.com/>

01 Windows

Standalone Installer를 다운받아 설치해주세요

별도의 설정 없이 계속 다음으로 설치하셔도 됩니다



Git 설치하기

홈페이지를 방문해서 Git을 설치하세요

<https://git-scm.com/>

02 macOS

Homebrew를 이용해 설치하세요

```
brew install git
```

Install

Latest version: 2.55.0 ([Release Notes](#))

Windows

macOS

Linux

Build from Source

There are several options for installing Git on macOS. Note that any non-source distributions are provided by third parties, and may not be up to date with the latest source release.

Choose one of the following options for installing Git on macOS:

Homebrew

Install **homebrew** if you don't already have it, then:

```
$ brew install git
```

MacPorts

Install **MacPorts** if you don't already have it, then:

```
$ sudo port install git
```

Xcode Command Line Tools

Apple ships a binary package of Git with **Xcode Command Line Tools**. You can install this via:

```
$ xcode-select --install
```

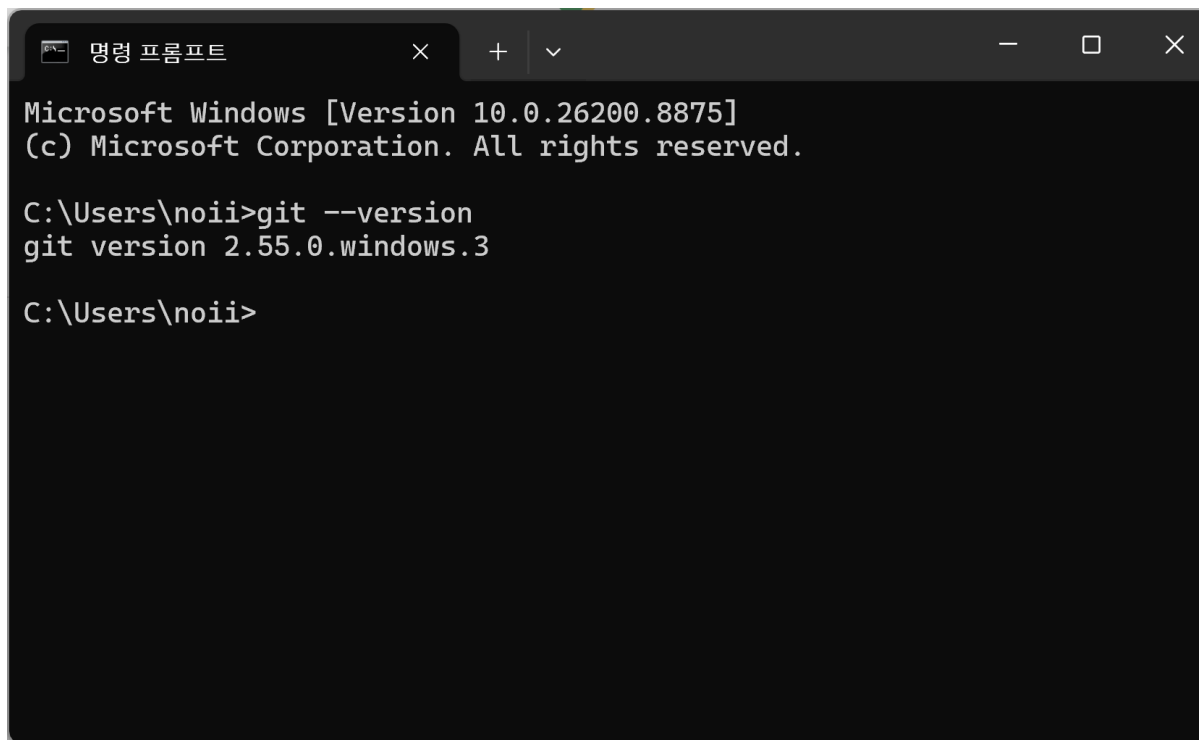
Binary installer

Tim Harper provided an installer for Git until version 2.33.0 / 2021. These installers are no longer linked from here because there are no updates since that version, nor are there plans to provide any.

Git 설치 확인하기

터미널(명령프롬프트) 내에서 다음 명령어를 확인해보세요
설치되어 있는 git의 버전을 확인하는 명령어 입니다

```
git --version
```



```
명령 프롬프트
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\noi>git --version
git version 2.55.0.windows.3

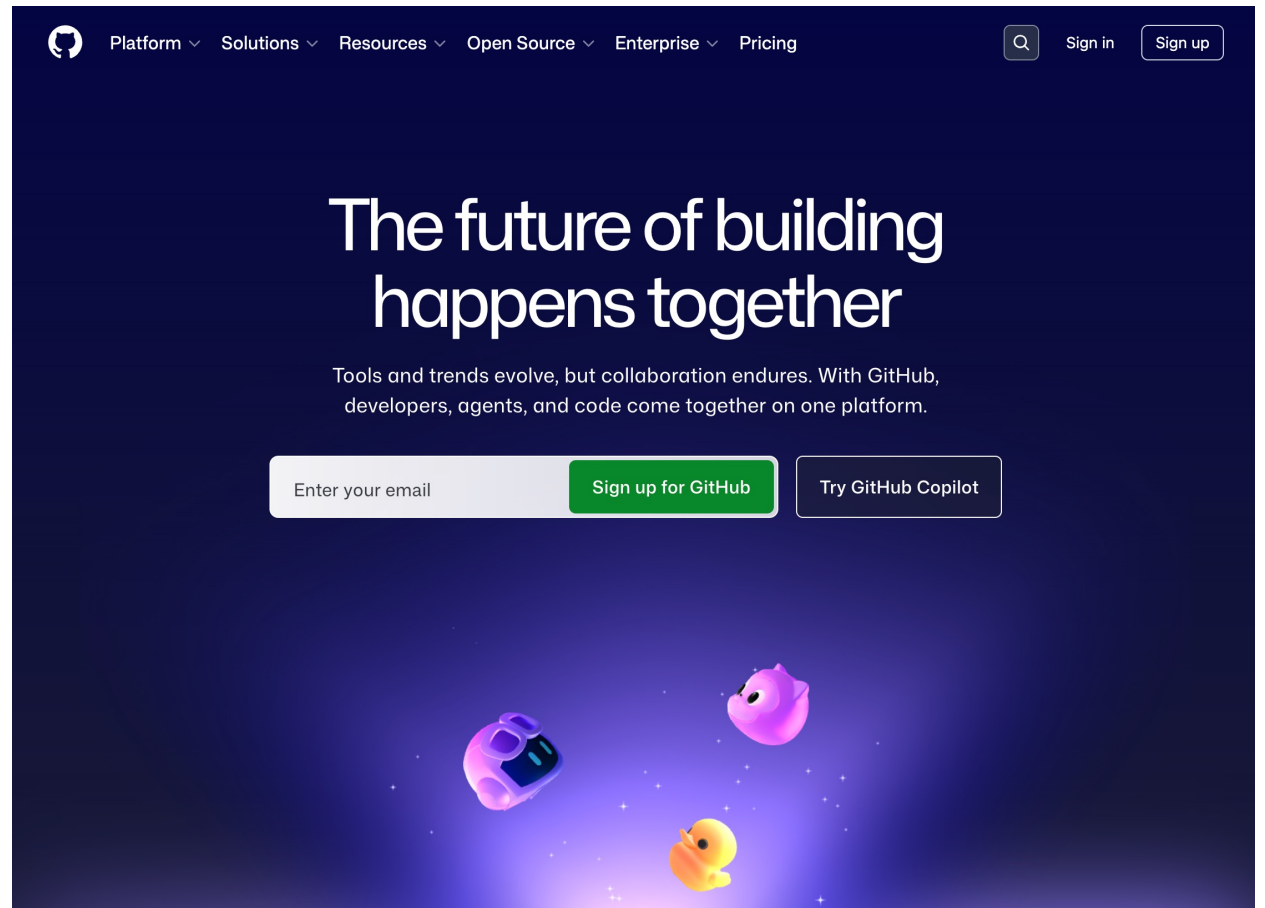
C:\Users\noi>
```

INSTALL

GitHub 가입하기

홈페이지를 방문에서 계정을 만드세요

<https://github.com/>

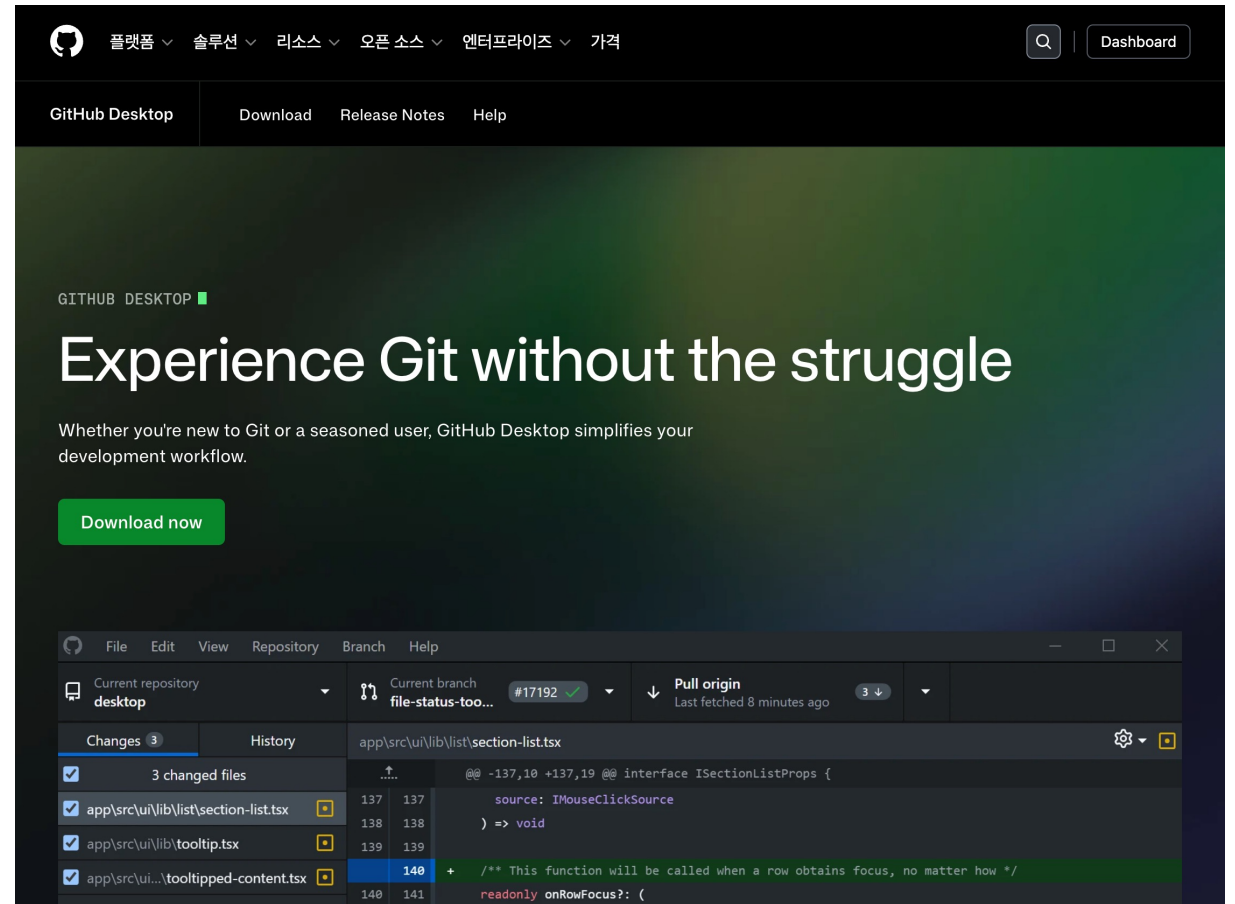


INSTALL

GitHub Desktop 설치하기

홈페이지를 방문해서 GitHub Desktop을 설치하세요

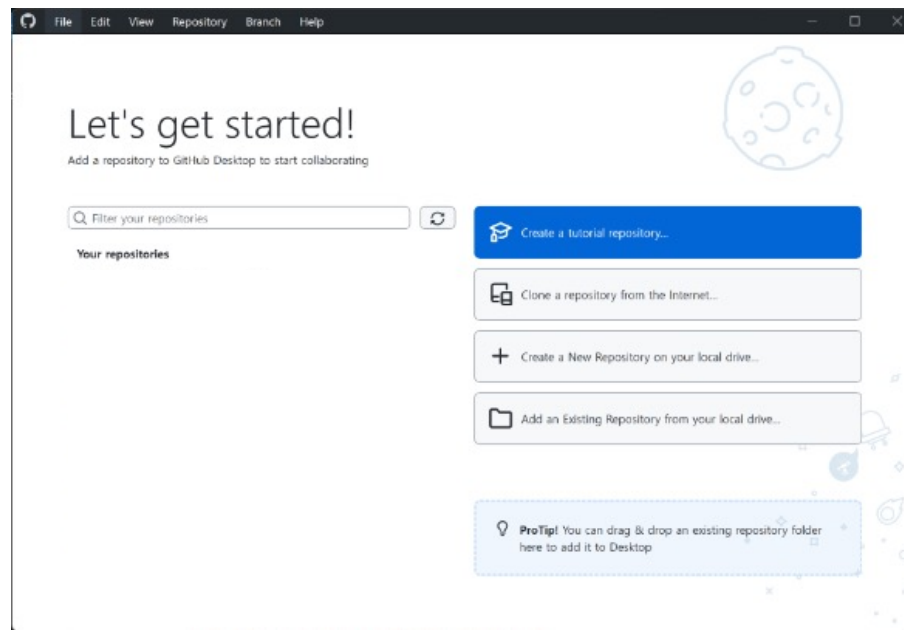
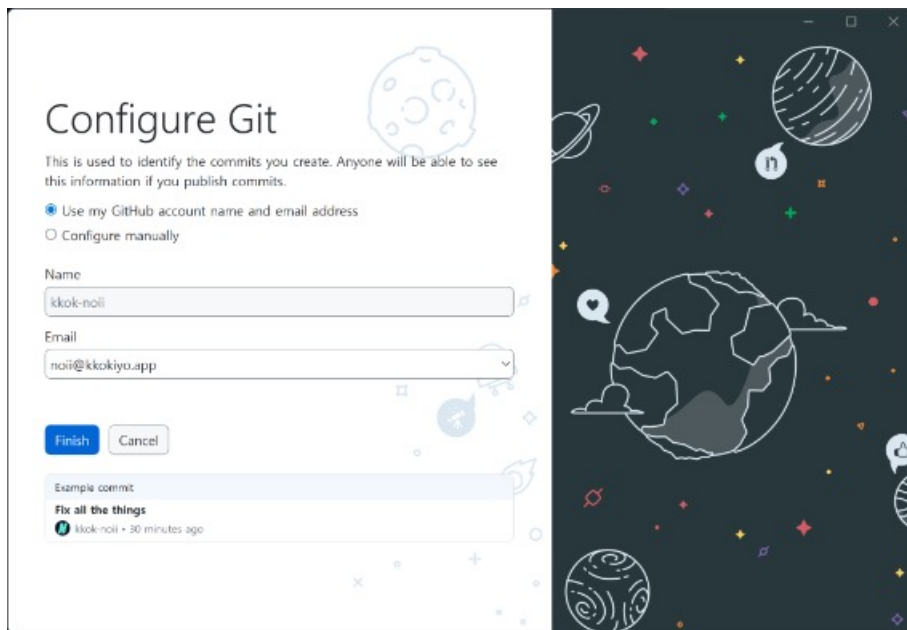
<https://github.com/apps/desktop>



INSTALL

GitHub Desktop 설치하기

Github 계정으로 로그인해주세요

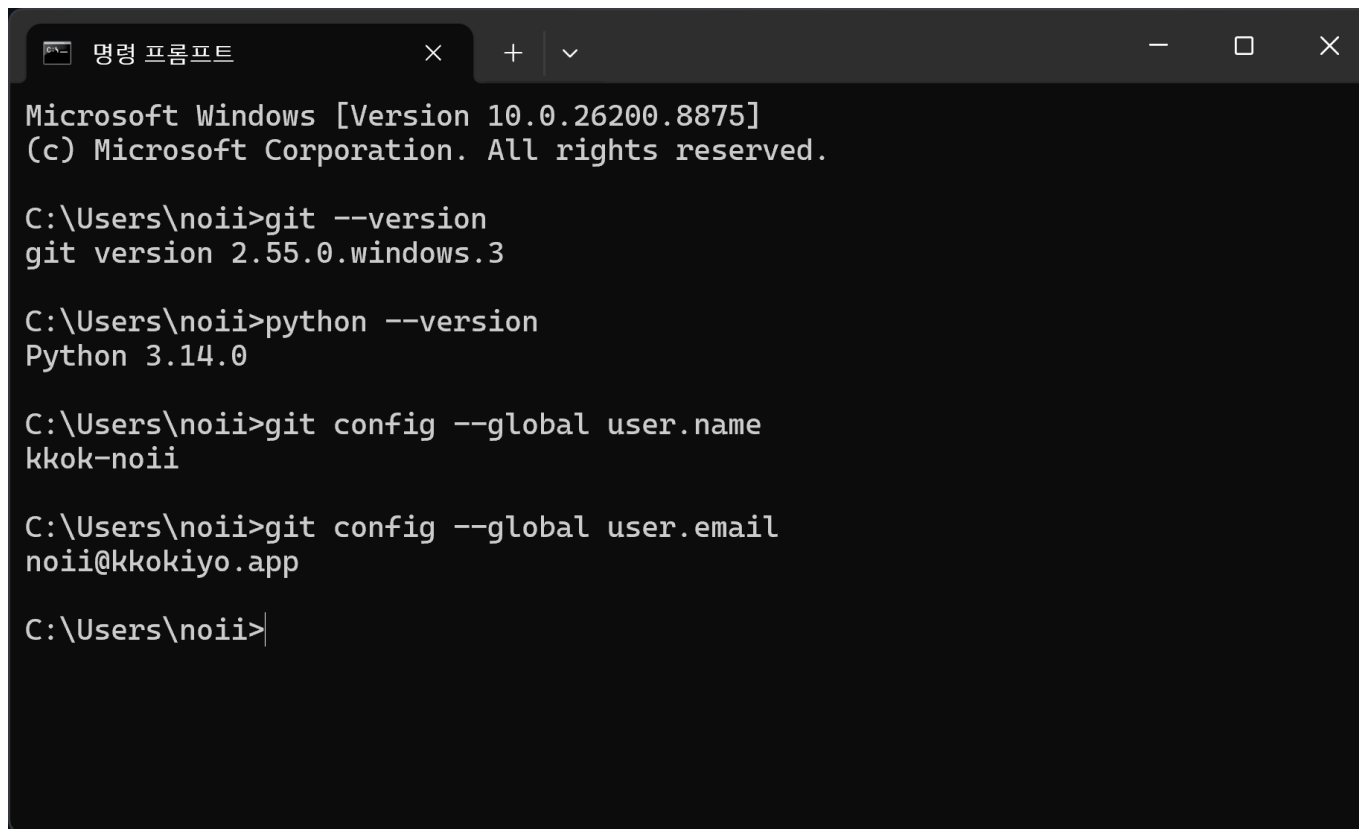


확인

명령어로 확인하기

터미널에서 아래 명령어를 실행시켜보세요.
이름·이메일이 GitHub 계정과 같은지 확인하세요.
다르면 정상적으로 기록되지 않아요.

```
git --version  
python --version  
git config --global user.name  
git config --global user.email
```



```
명령 프롬프트  
Microsoft Windows [Version 10.0.26200.8875]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\noi>git --version  
git version 2.55.0.windows.3  
  
C:\Users\noi>python --version  
Python 3.14.0  
  
C:\Users\noi>git config --global user.name  
kkok-noii  
  
C:\Users\noi>git config --global user.email  
noi@kkokiyo.app  
  
C:\Users\noi>
```

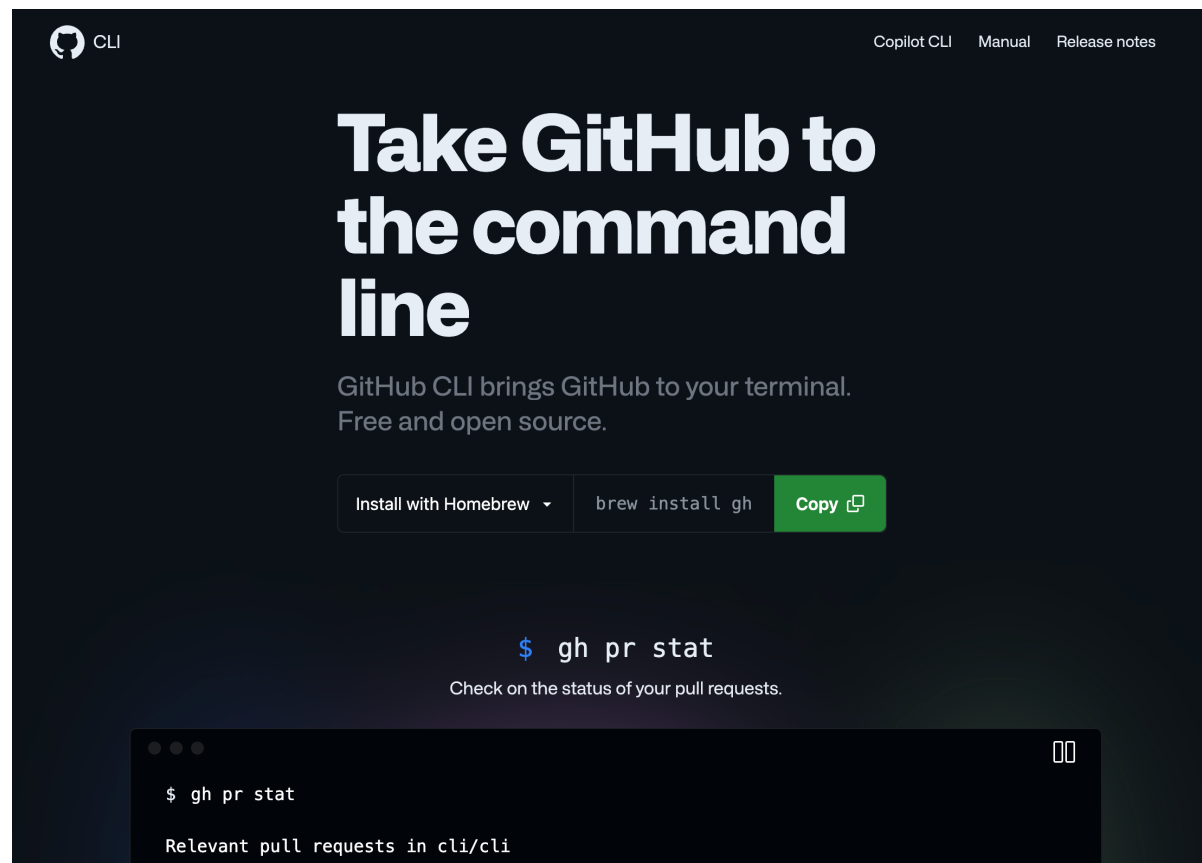
INSTALL

GitHub CLI 설치하기

홈페이지를 방문해서 GitHub CLI를 설치하세요

<https://cli.github.com/>

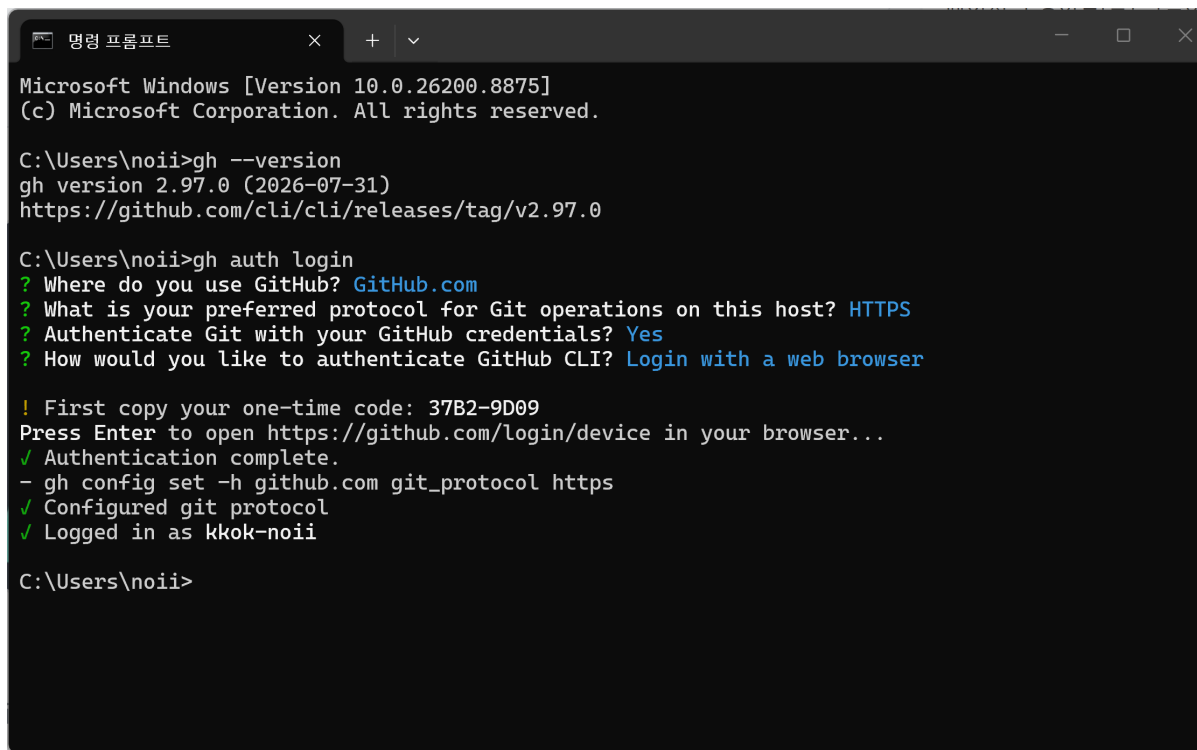
```
# Windows  
winget install --id GitHub.cli  
  
# macOS  
brew install gh
```



GitHub CLI 설치하기

다음 명령어로 설치 확인 및 로그인을 진행해주세요

```
gh --version  
gh auth login
```



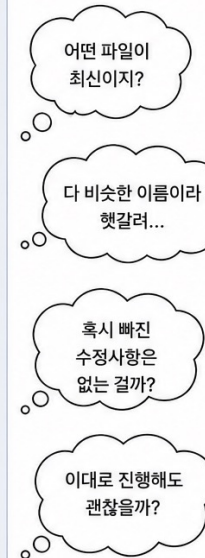
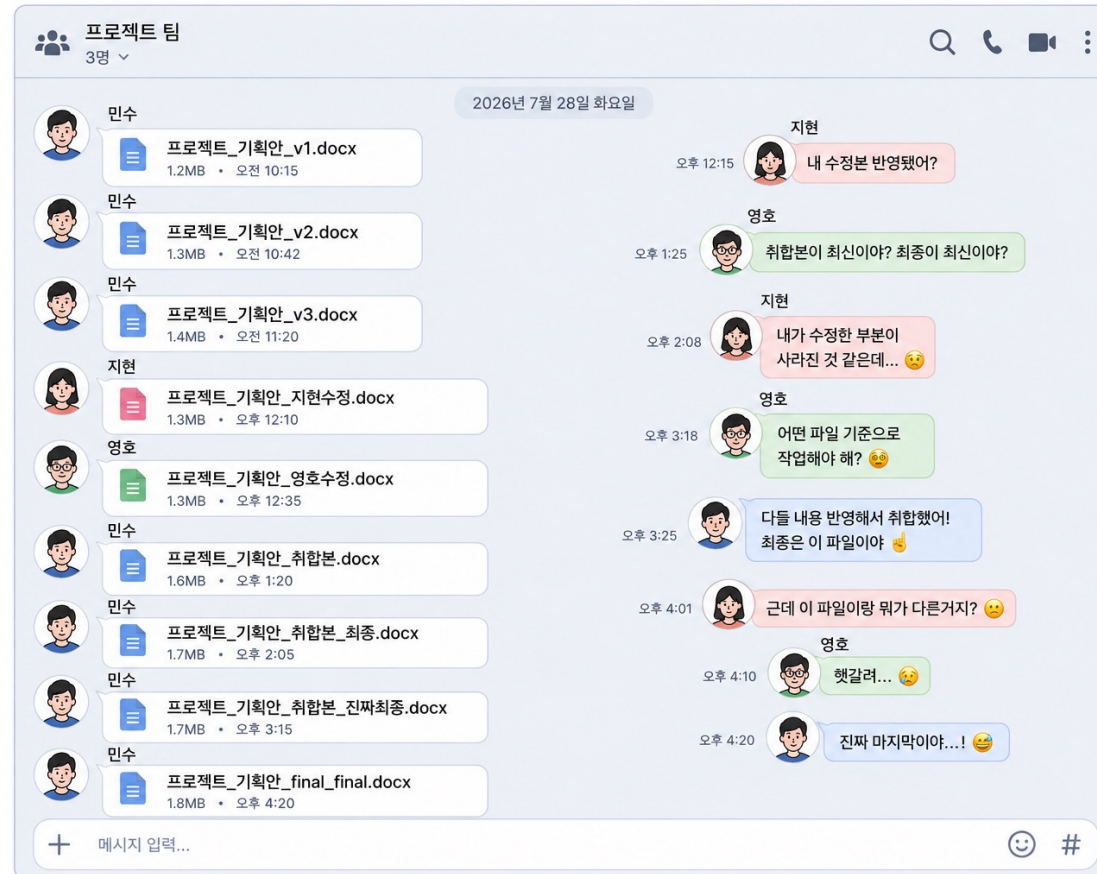
```
명령 프롬프트  
Microsoft Windows [Version 10.0.26200.8875]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\noi>gh --version  
gh version 2.97.0 (2026-07-31)  
https://github.com/cli/cli/releases/tag/v2.97.0  
  
C:\Users\noi>gh auth login  
? Where do you use GitHub? GitHub.com  
? What is your preferred protocol for Git operations on this host? HTTPS  
? Authenticate Git with your GitHub credentials? Yes  
? How would you like to authenticate GitHub CLI? Login with a web browser  
  
! First copy your one-time code: 37B2-9D09  
Press Enter to open https://github.com/login/device in your browser...  
✓ Authentication complete.  
- gh config set -h github.com git_protocol https  
✓ Configured git protocol  
✓ Logged in as kkok-noii  
  
C:\Users\noi>
```

SESSION 2

왜 Git인가

익숙한 장면

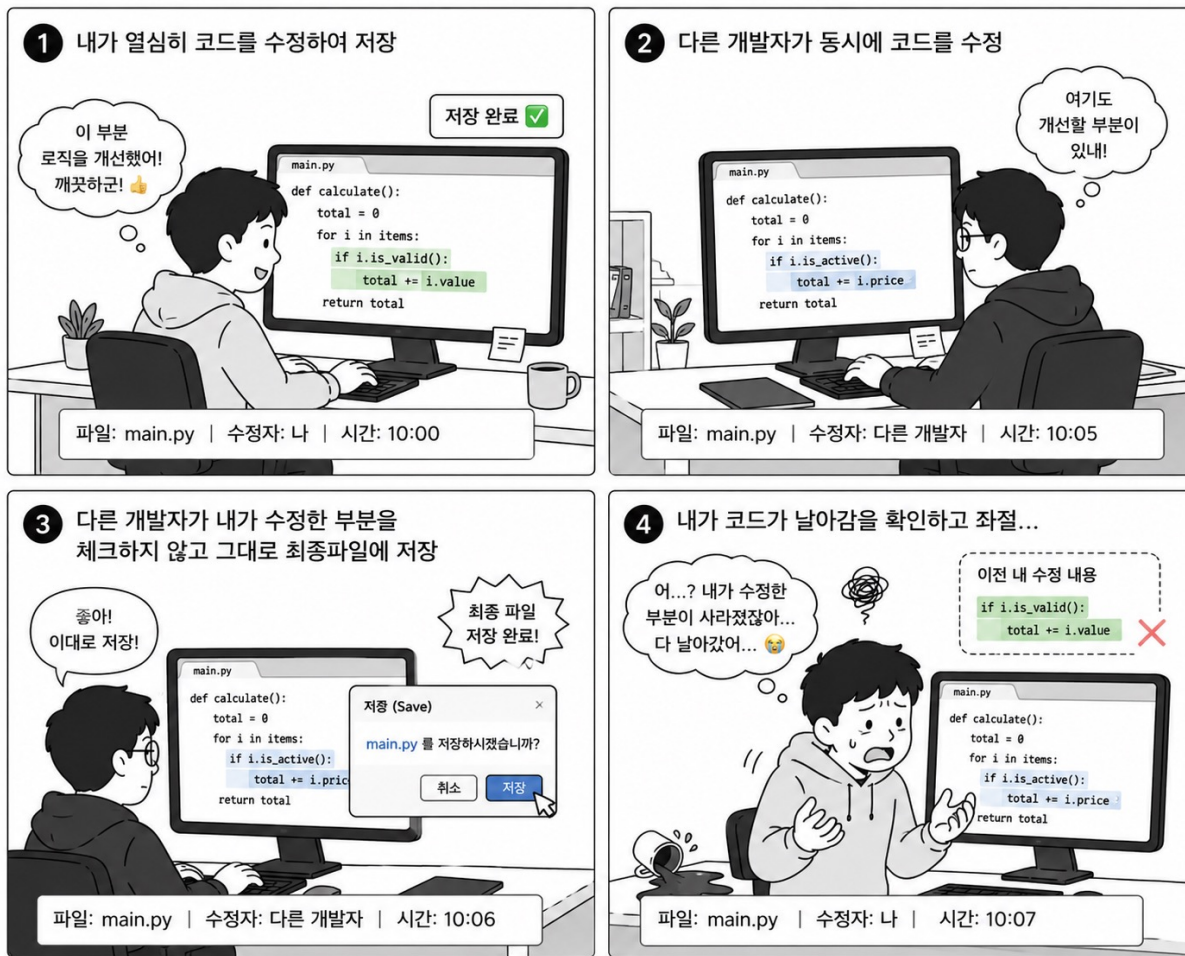
어느 게 최종인가요



그 내용은 취소됐어요



제 코드가 사라졌어요



소프트웨어 형상관리

SW 개발 및 유지보수 과정에서 발생하는 **모든 산출물(형상)**을 정의

이들 형상에 대한 변경을 체계적으로 관리, 제어하는 활동

각 산출물에 대하여 변경 내용이 있을 때 마다 **버전을 기록**하고 **작성자, 변경 내역** 등 상세 기록

개발환경

소스코드

문서

제품

Git

컴퓨터 파일의 변경 이력을 추적하고 여러 사용자가 작업을 협업할 수 있도록 돕는 **분산 버전 관리 시스템**

SCM을 수행하기 위한 소스코드 관리 도구

Git의 주요 기능

- 01 이력 기록 및 추적: 모든 파일에 대하여 변경자, 변경 내용 기록
- 02 원격 저장소 및 공유: 로컬 저장소, 원격 저장소간 협업 가능
- 03 변경 이력 병합: 동일 파일에 대한 두건 이상의 수정 발생 시, 통합 기능 제공

<https://github.com/>

Github

Git 저장소 호스팅 서비스: 소스코드를 온라인상에 저장 및 관리할 수 있는 플랫폼

협업 도구: 여러 개발자가 같은 프로젝트에 참여하여 버전관리 & 코드 리뷰 가능

Github의 주요 기능

- 01 Git 기반의 버전 관리
- 02 Pull Request를 통한 코드 리뷰
- 03 Issues로 할 일, 버그 관리
- 04 Actions로 자동 배포 & 테스트

SESSION 3

첫 저장소, 첫 커밋

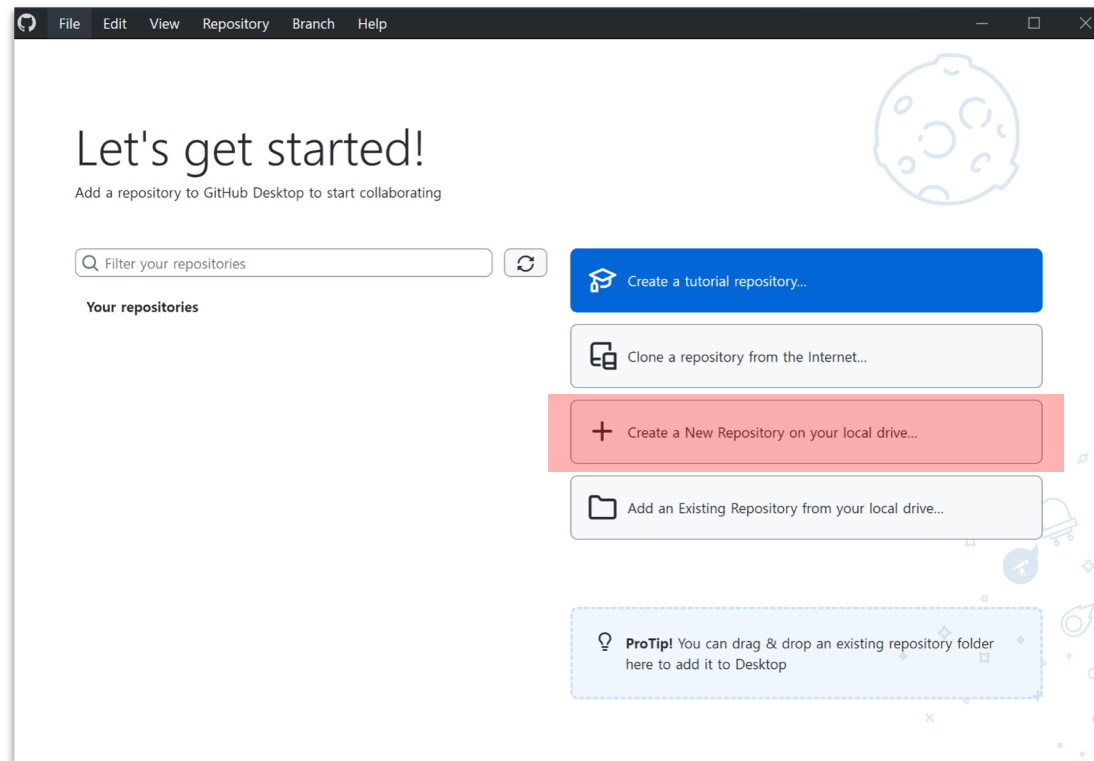
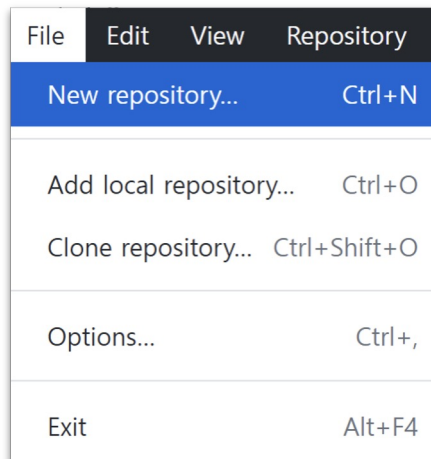
Repository 만들기

Repository

버전 관리를 시작한 프로젝트 폴더

모든 파일의 변경 이력, 히스토리를 기록

히스토리를 기반으로 비교하여 변경사항 추적 가능



Repository 만들기

01 **Name** calc_project

해당 Repository 이름

02 **Description** 계산기를 만듭시다

해당 Repository에 대한 설명글
선택사항이나 실습을 위해 입력해보세요

03 **Local Path**

해당 Repository가 생성될 경로 선택

해당 경로 하위에 [Name] 폴더 생성

* Windows의 경우, One Drive 외 위치에 진행해주세요

Create a new repository

Name
calc_project

Description
계산기를 만듭시다

Local path
C:\Users\noi\Documents\projects Choose...

☒ Initialize this repository with a README

Git ignore
Python

License
MIT License

The repository will be created at C:\Users\noi\Documents\projects\calc_project.

Create repository Cancel

Repository 만들기

04 Initialize this repository with a README

Repository 생성 시 README.md 파일 생성 여부

Github 페이지 하단 설명 글 파일

Markdown 문법으로 작성

README 구경하기

[Streamlit](#)

[Ponytail](#)

Create a new repository

Name
calc_project

Description
계산기를 만듭시다

Local path
C:\Users\noi\Documents\projects Choose...

☒ Initialize this repository with a README

Git ignore
Python

License
MIT License

The repository will be created at C:\Users\noi\Documents\projects\calc_project.

Create repository Cancel

Repository 만들기

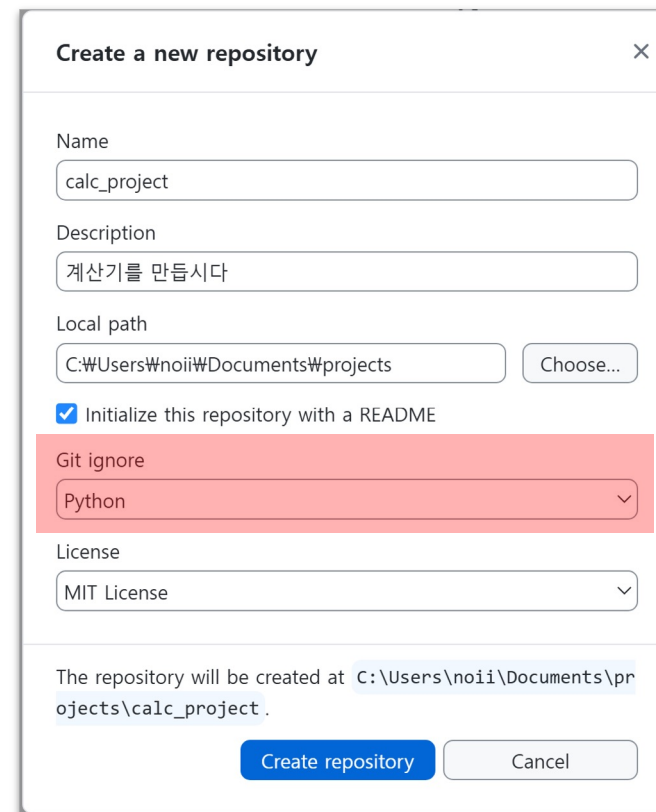
04 Git Ignore Python

.gitignore 파일 생성 및 기본 내용 선택

일반적으로 해당 프로젝트에서 사용할 개발 언어 선택

💡 .gitingore ?

해당 Repository에서 버전 기록이 필요없는 파일 목록 명시



Create a new repository

Name
calc_project

Description
계산기를 만듭시다

Local path
C:\Users\noi\Documents\projects Choose...

☒ Initialize this repository with a README

Git ignore
Python

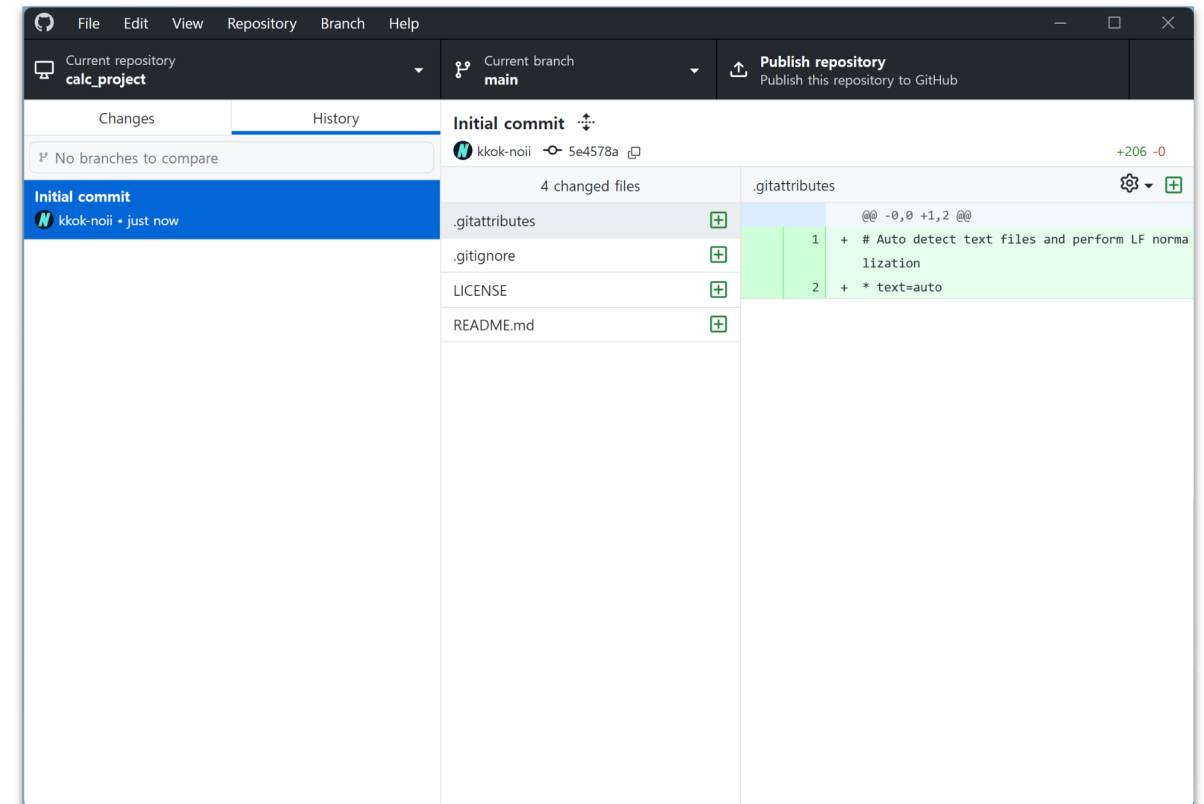
License
MIT License

The repository will be created at C:\Users\noi\Documents\projects\calc_project.

Create repository Cancel

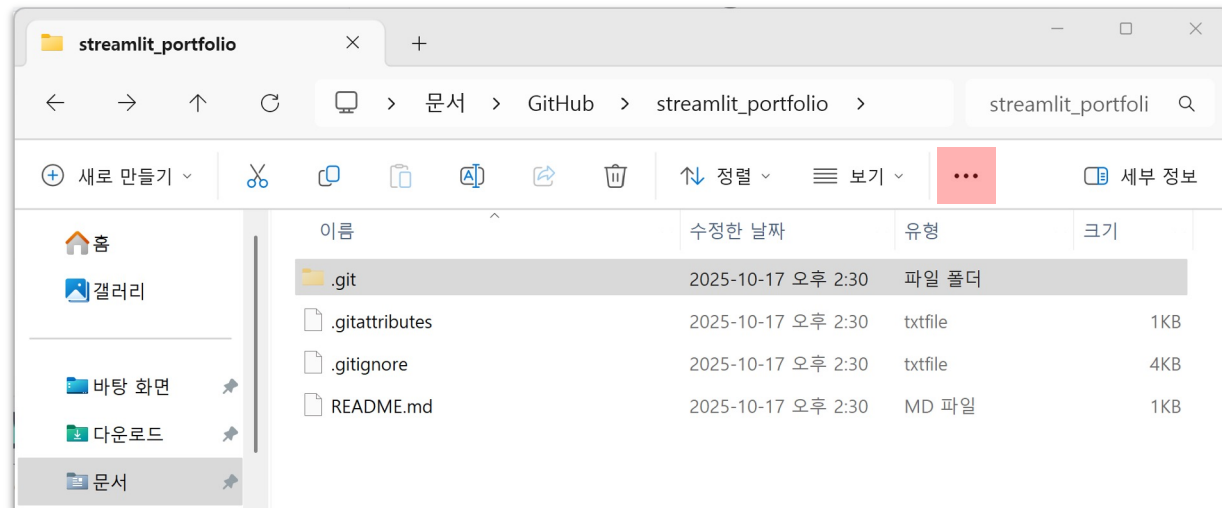
Repository 확인하기

Repository가 생성되며 현 상태를 첫 버전으로 기록합니다
첫 버전의 이름은 **Initial commit**



Repository 확인하기

프로젝트 생성 위치로 가서 숨김파일을 보면 숨김 폴더 **.git**이 있어요
프로젝트 폴더 내에 .git폴더가 있다 = Git으로 버전 관리 중
.git폴더를 지우면 변경 **내역, 히스토리 모두 삭제**



(윈도우: 해당 폴더 ... > 옵션 > 보기 > 숨김 파일 및 폴더 > 숨김 파일, 폴더 및 드라이브 표시 체크)

(Mac Finder에서는 ⌘(Command) + ⌘(Shift) + . 로 숨김 파일 확인이 가능합니다)

Commit

커밋(commit)이란?

작업한 변경사항을 하나의 단위로 저장하는 것



커밋을 하면, 현재 파일의 상태를 스냅샷처럼 기록하고 이후에 언제든지 돌아가거나 비교할 수 있습니다.



커밋 메시지 예시

```
git commit -m "로그인 기능 구현"
```

의미

지금까지의 변경사항을 '로그인 기능 구현'이라는 이름으로 저장

1 코드 수정

파일을 수정하면 변경사항이 생깁니다.

```
login.py
1 def login(user, pw):
2     if user == "admin" and pw == "1234":
3         return True
4     else:
5         return False
```

수정!

작업 중인 파일 (Working Directory)

2 변경사항 스테이징

커밋할 변경사항을 준비합니다.

Staging Area
login.py
(수정됨)

git add login.py

3 커밋 생성

변경사항을 하나의 단위로 저장합니다.

Commit
"로그인 기능 구현"
(2024-05-26 10:30)

git commit -m "로그인 기능 구현"

4 커밋 히스토리

커밋들이 순서대로 기록됩니다.

로그인 기능 구현
(2024-05-26 10:30)
회원가입 기능 구현
(2024-05-25 16:20)
프로젝트 초기 설정
(2024-05-24 09:15)
...

커밋을 사진 찍기에 비유하면?



작업 중
(어질러진 책상)



스테이징
(사진 찍을 준비)



커밋
(사진 촬영!)



히스토리
(시간 순서대로 앨범 저장)

★ 왜 커밋이 중요할까?

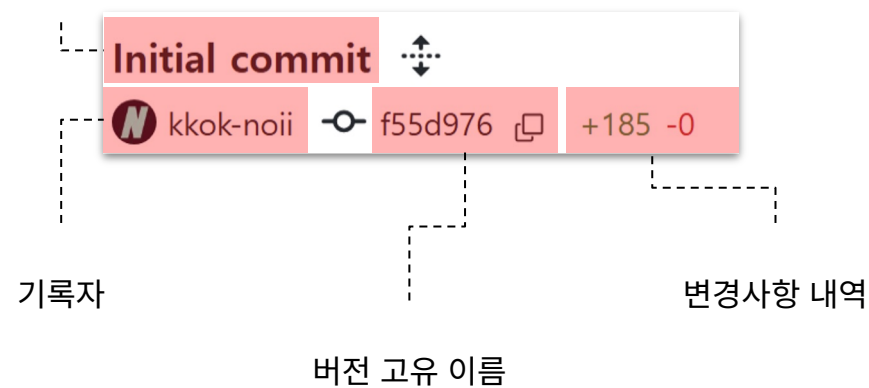
- ✓ 변경사항을 안전하게 기록
- ✓ 이전 상태로 돌아갈 수 있음
- ✓ 변경 이력을 팀원과 공유
- ✓ 문제 발생 시 원인 파악이 쉬움

Commit

현재까지의 Repository 내 변경사항을 새로운 버전으로 저장하는 행위

- 01 새롭게 추가된 파일
- 02 삭제된 파일
- 03 내용이 변경된 파일(변경된 부분)

Commit 이름



Commit 해보기

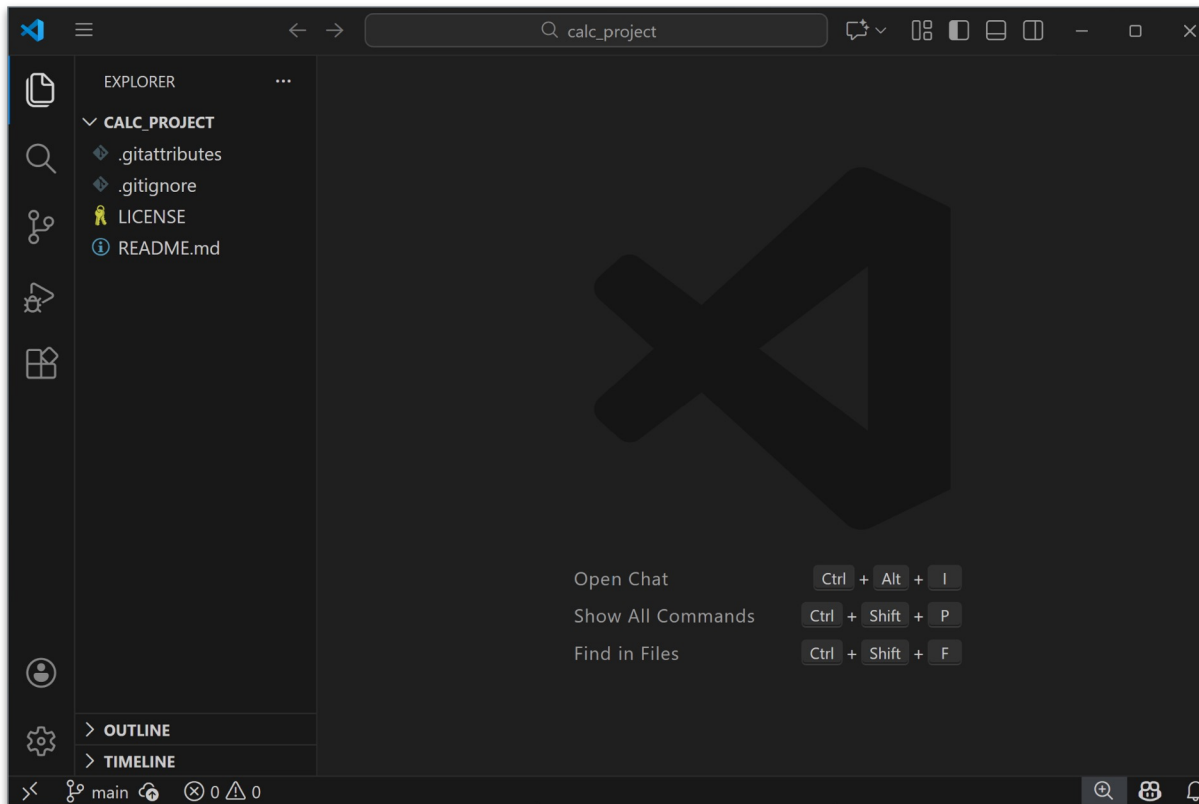
계산기 프로젝트를 Python으로 구현하며 변경 이력을 남겨봅시다
다음과 같은 순서로 작업을 진행해봅시다

1. 더하기 함수 만들기
2. 빼기 함수 만들기
3. 곱하기 함수 만들기
4. 나누기 함수 만들기

Commit 해보기

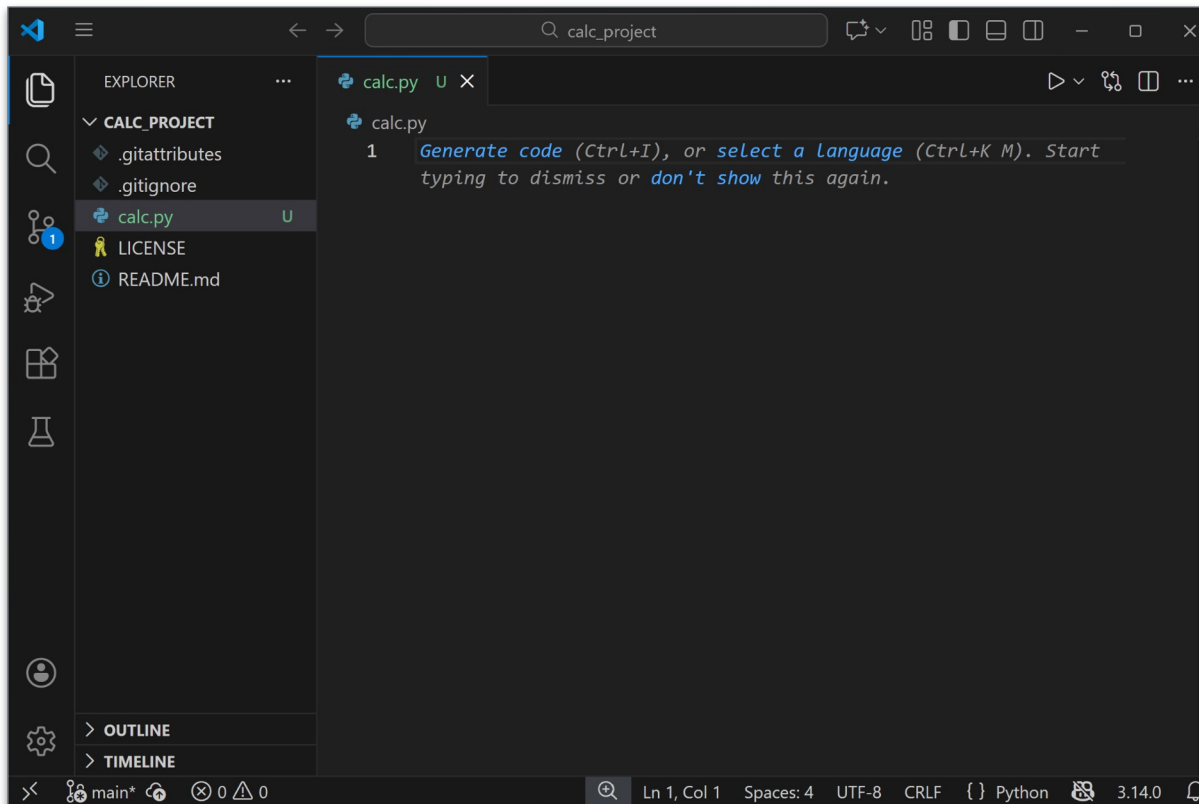
VS Code로 생성한 Repository 열기

파일 > 폴더 열기



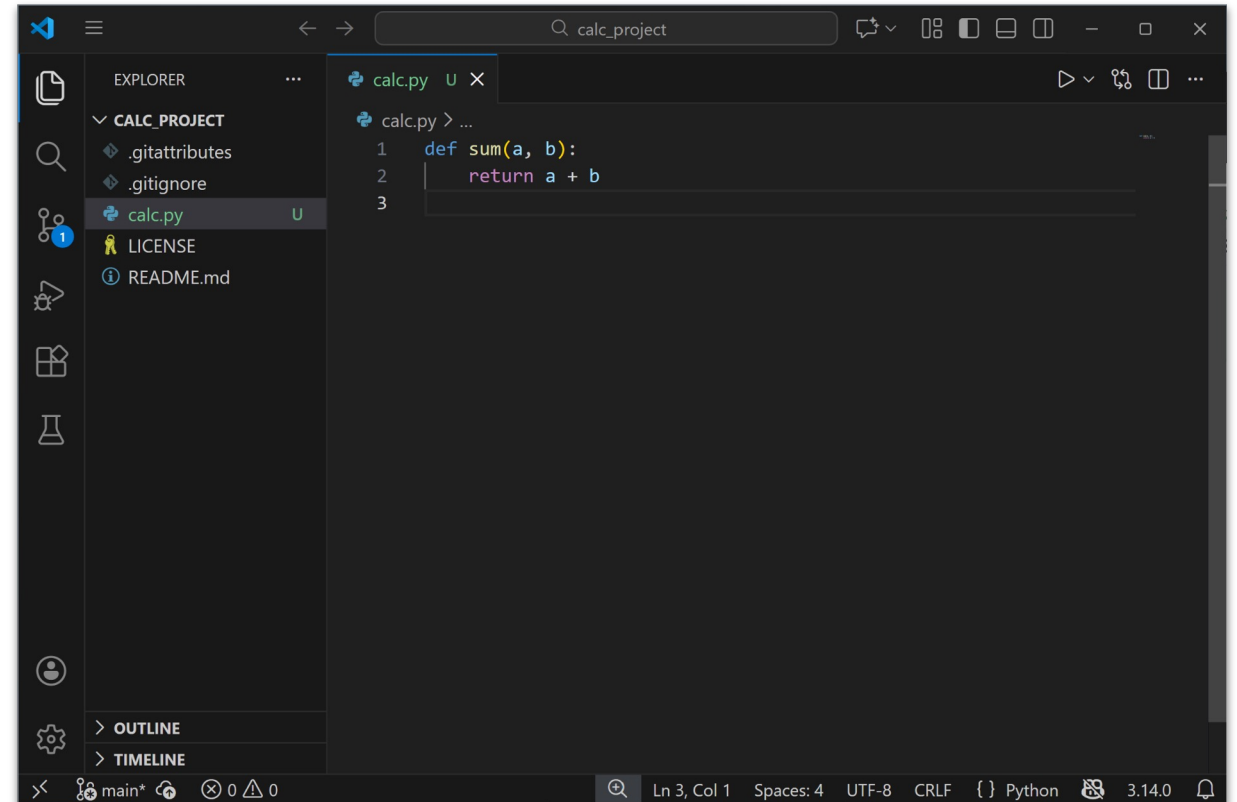
Commit 해보기

프로그래밍 시작을 위해 calc.py 파일 생성하기
새로 생긴 파일 옆에 **U(Untracked)** 확인



Commit 해보기

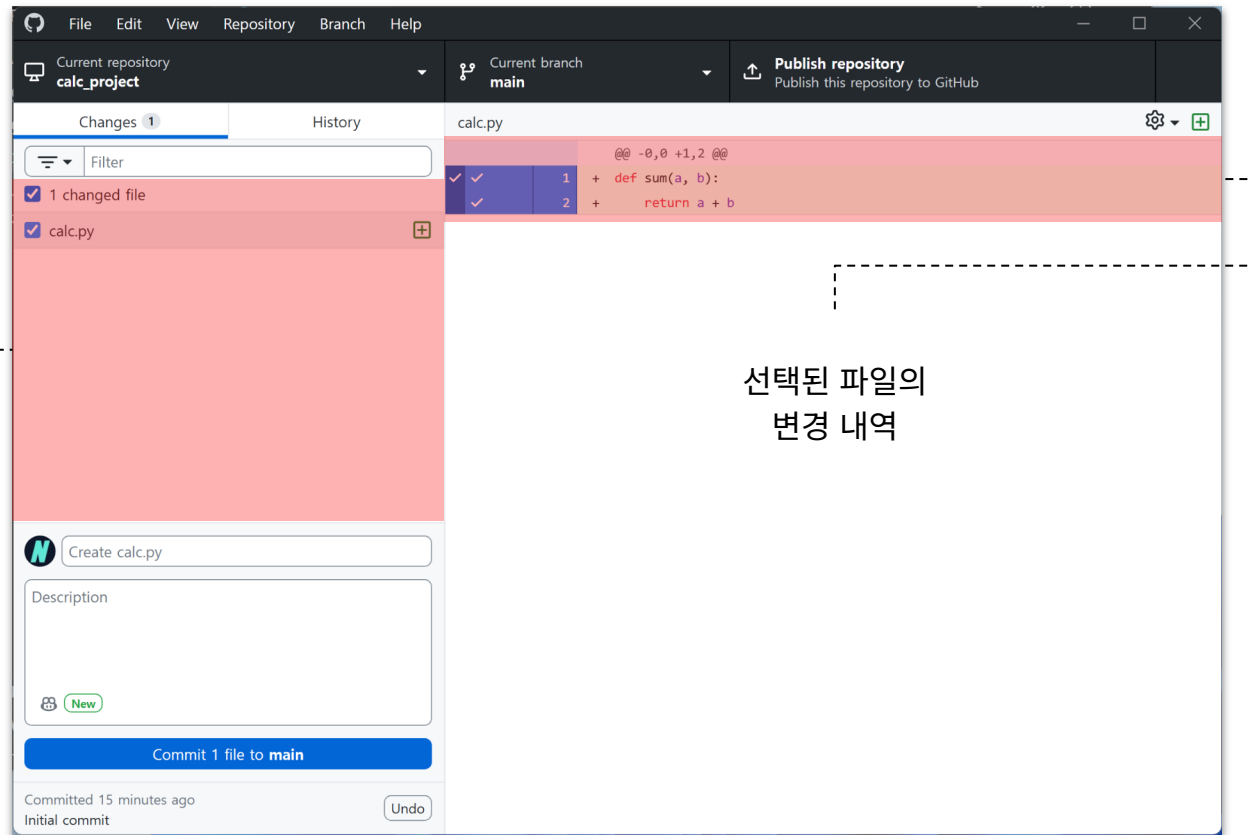
더하기, sum 함수 구현하기



Commit 해보기

Github Desktop에서 변경 사항 확인하기

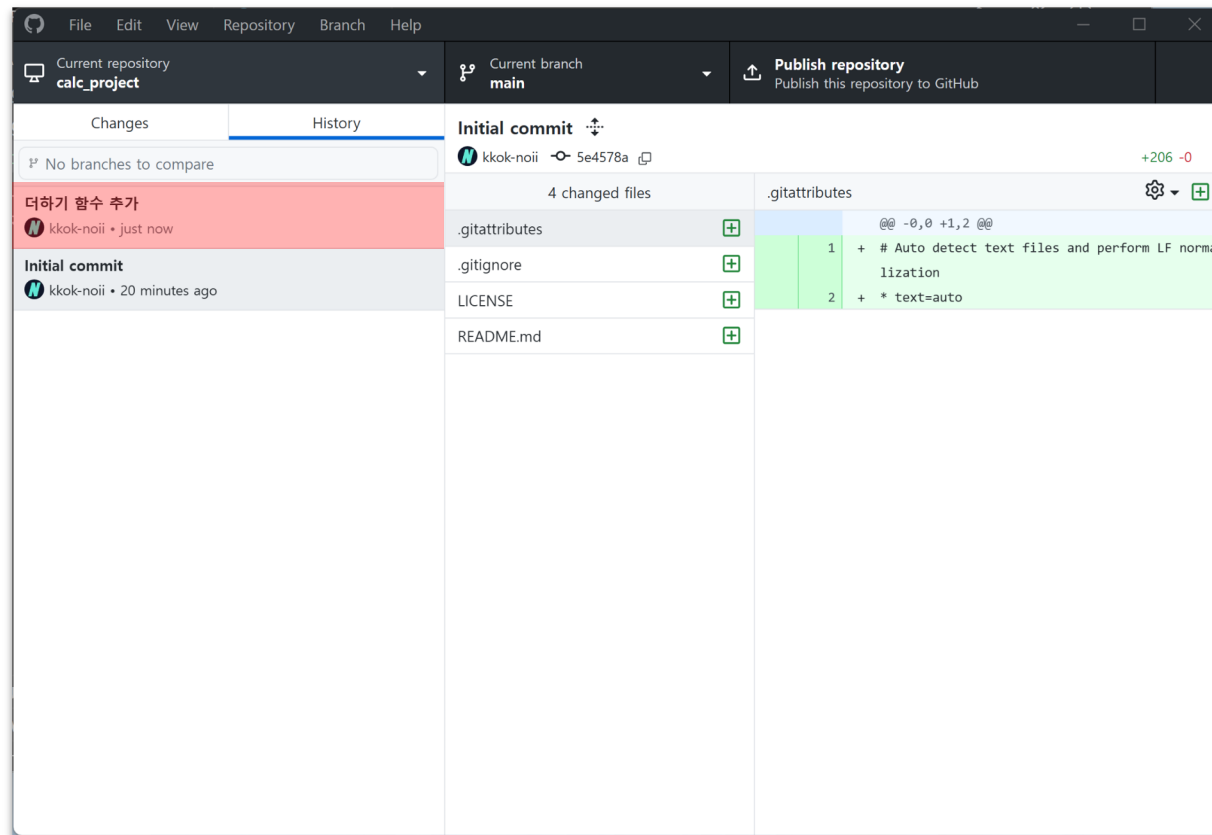
변경된 파일 리스트



선택된 파일의
변경 내역

Commit 해보기

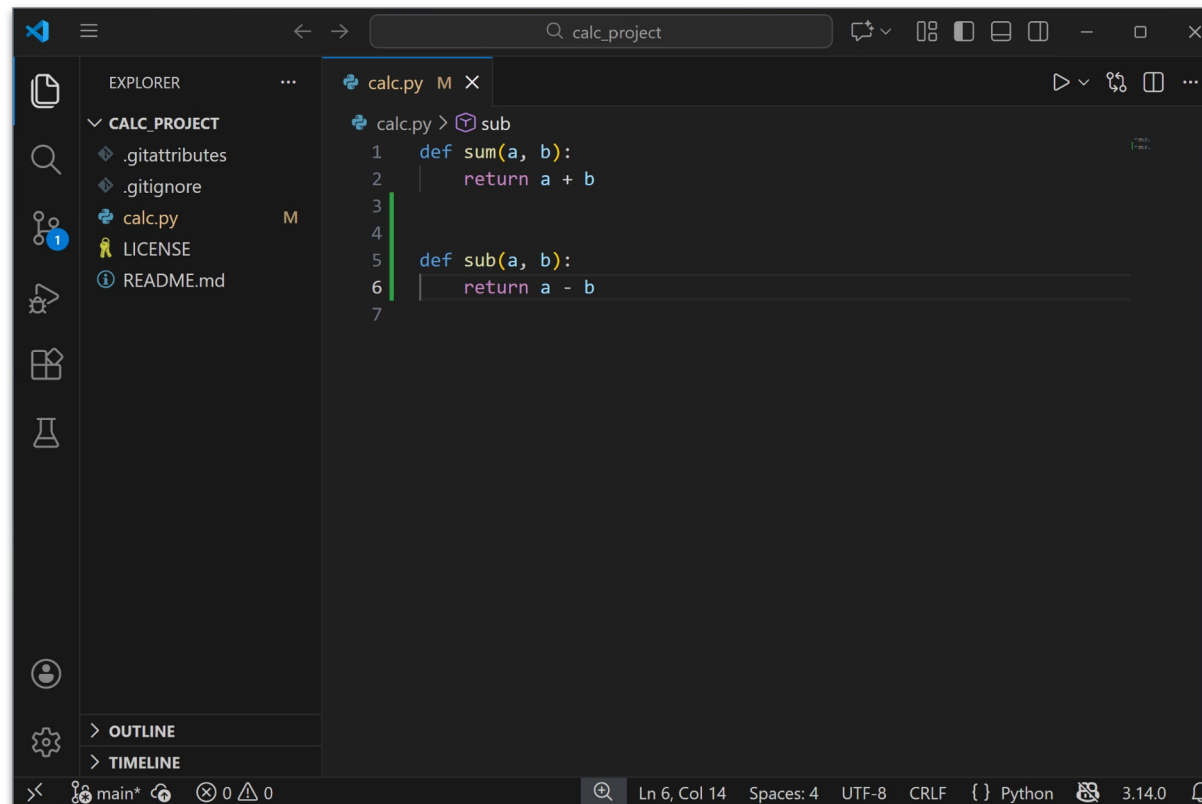
더하기 함수 추가 제목으로 Commit 하기
첫 Commit 완료



Commit 해보기

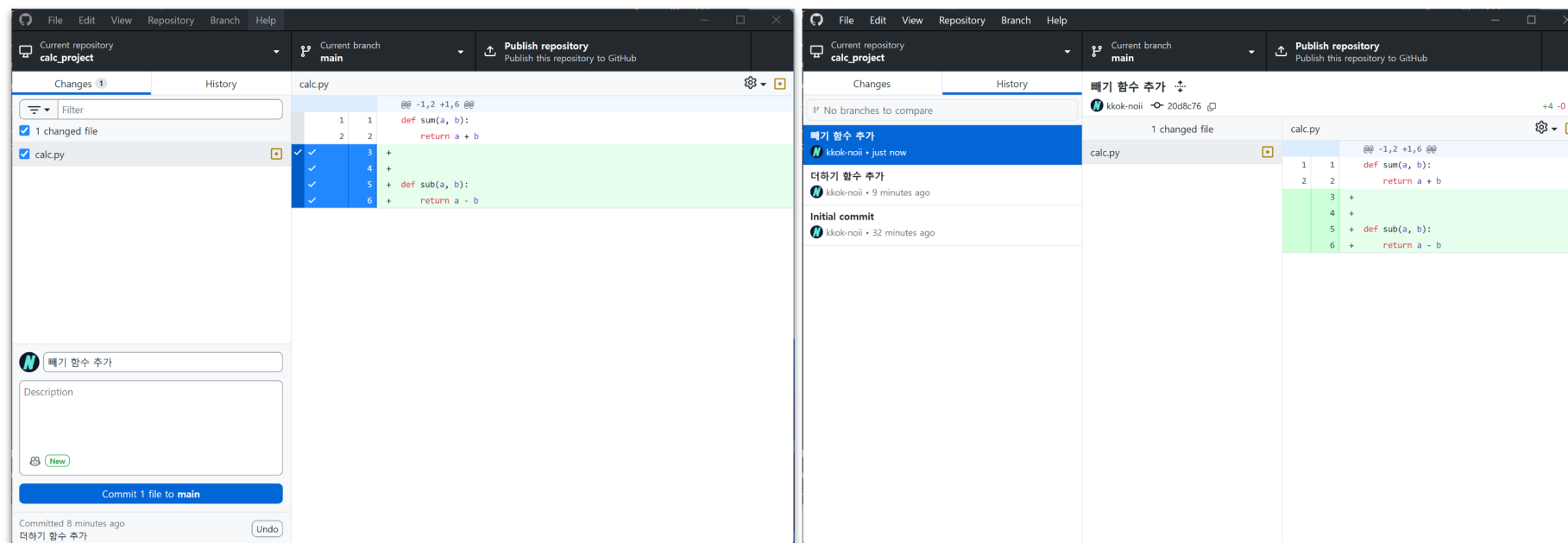
빼기, sub 함수 구현

수정된 파일 옆에 **M(Modified)** 확인



Commit 해보기

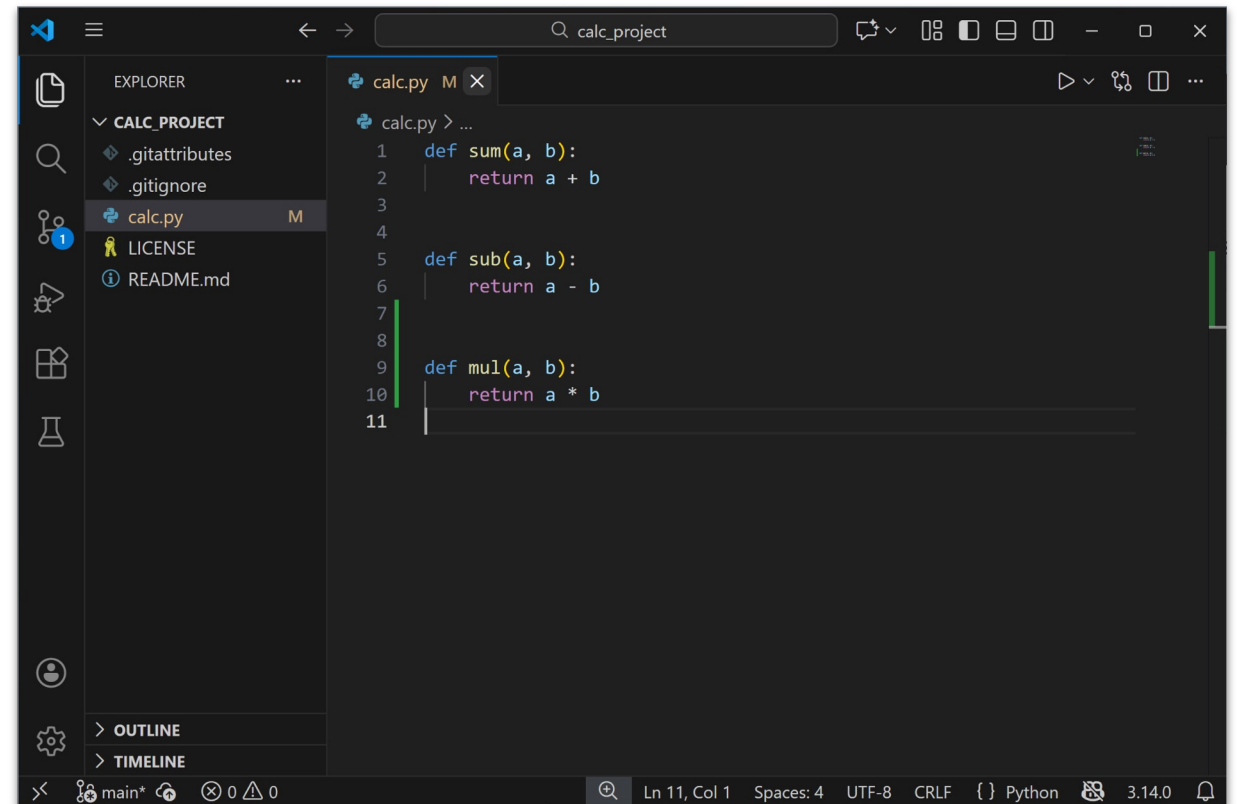
빼기 함수 추가 제목으로 Commit 하기



Commit 해보기

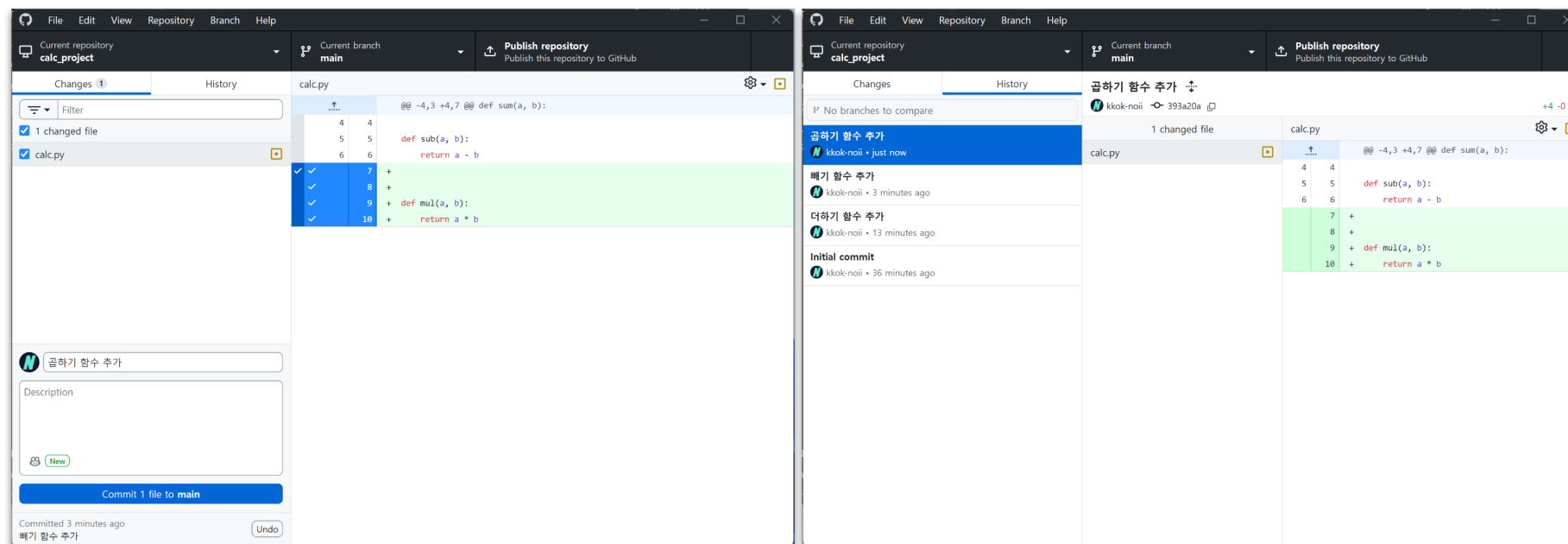
곱하기, mul 함수 구현

수정된 파일 옆에 **M(Modified)** 확인



Commit 해보기

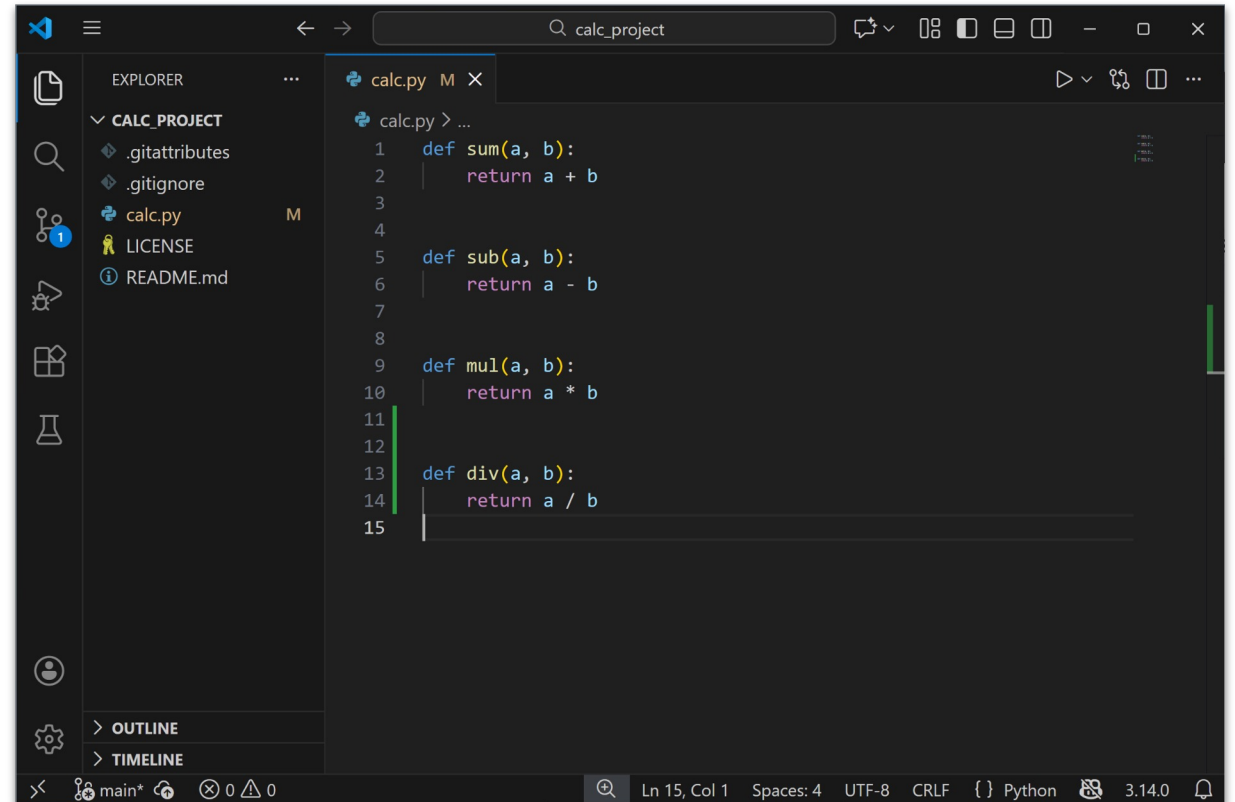
곱하기 함수 추가 제목으로 Commit 하기



Commit 해보기

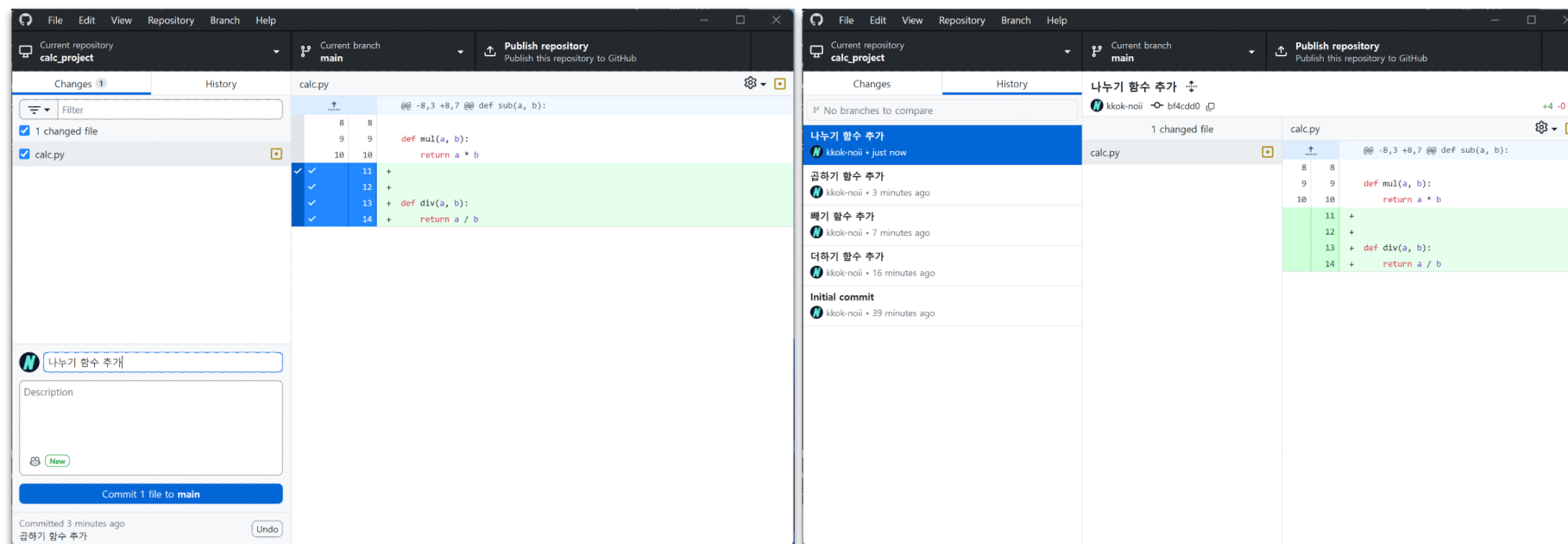
나누기, div 함수 구현

수정된 파일 옆에 **M(Modified)** 확인



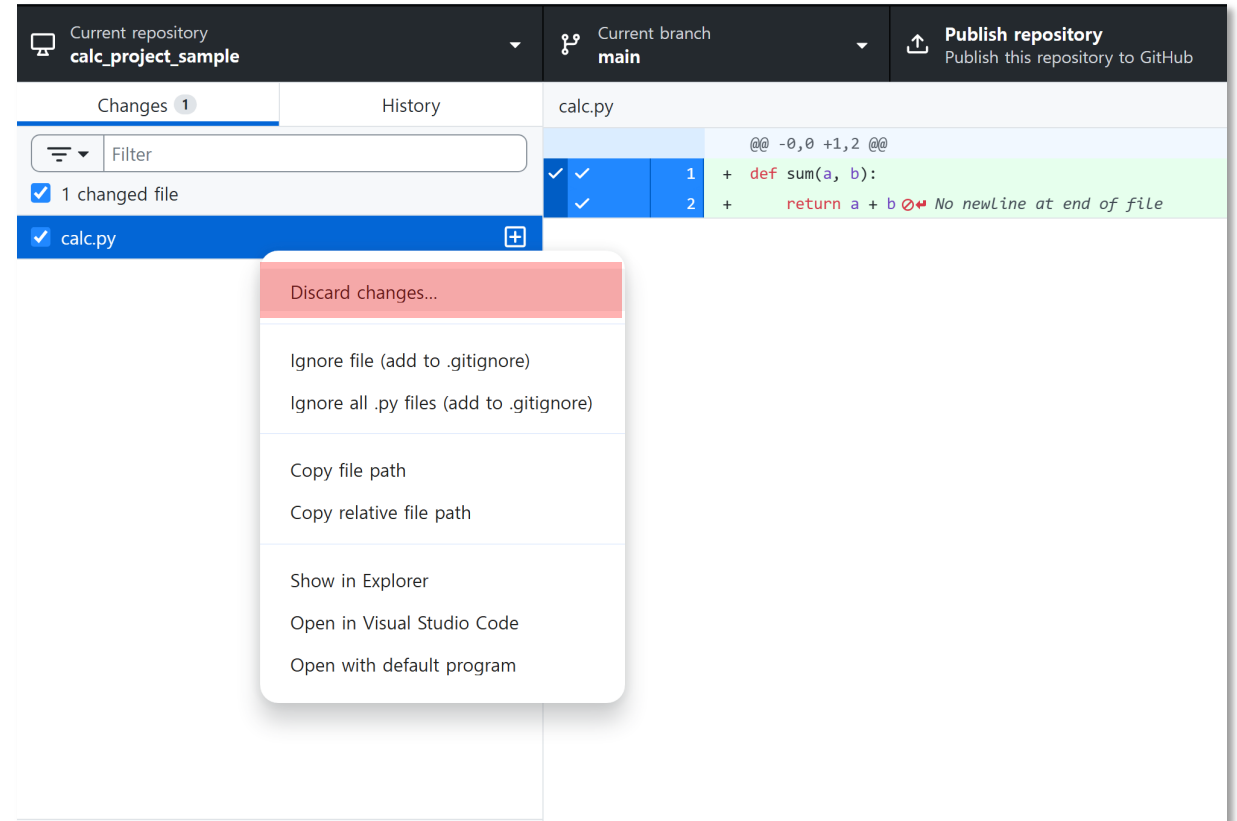
Commit 해보기

나누기 함수 추가 제목으로 Commit 하기



Commit 취소해보기

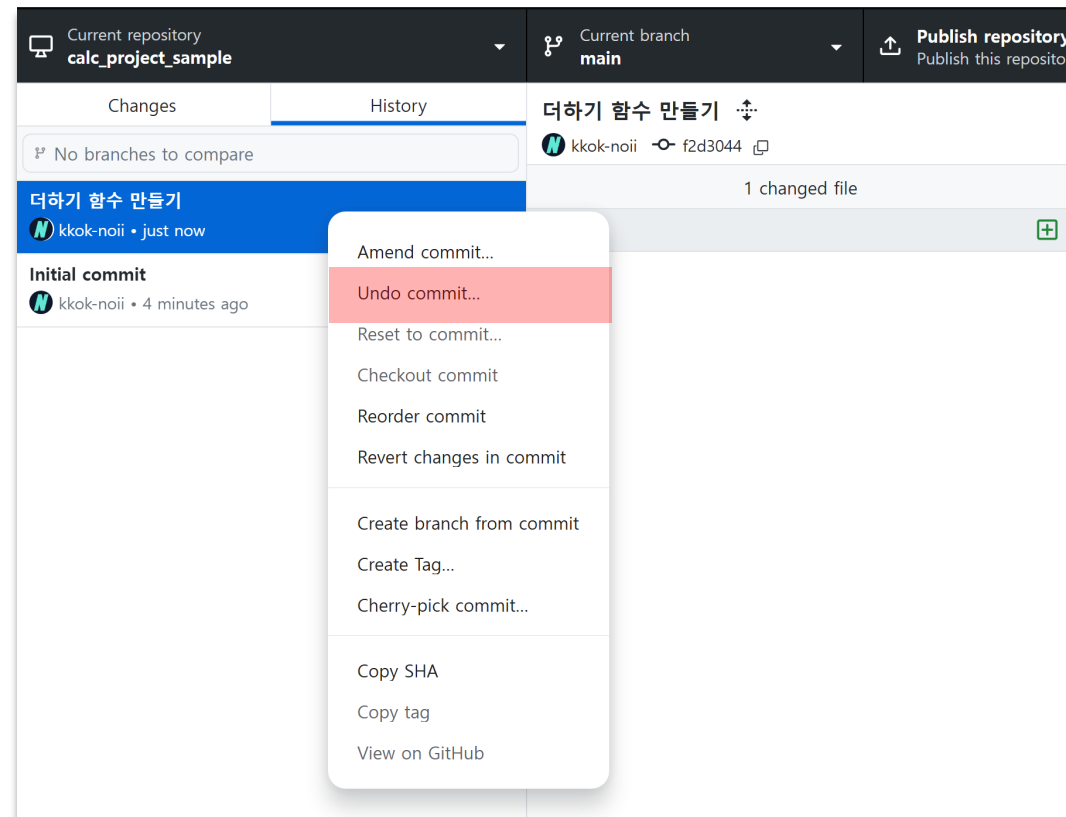
아직 Commit 되지 않은 내용은 Discard changes로 취소 가능



Commit 취소해보기

가장 최근 Commit은 Undo commit으로 취소 가능
작업한 내용은 Commit 안된 상태로 변경

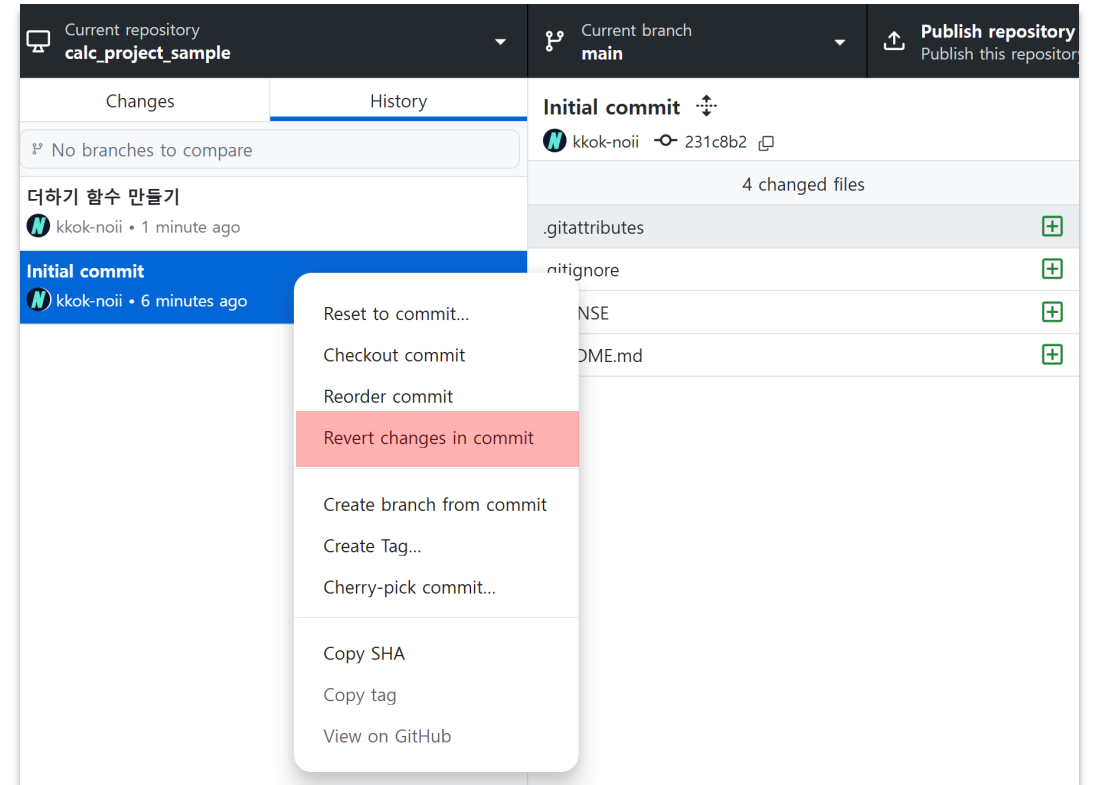
Amend commit: 해당 Commit 내용 수정



Commit 취소해보기

옛 상태로 되돌릴 때 Revert changes in commit

기록들을 지우는게 아니라 옛 상태를 다시 최신 버전으로 기록



SESSION 4

커밋 습관과 .gitignore

규칙

메시지는 태그 + 무엇을 왜

이렇게

[feat] 나누기 0 예외 처리 추가

[fix] 소수점 반올림 오류 수정

[docs] README 실행 방법 추가

이렇게는 말고

수정

asdf

최종

커밋 단위의 기준

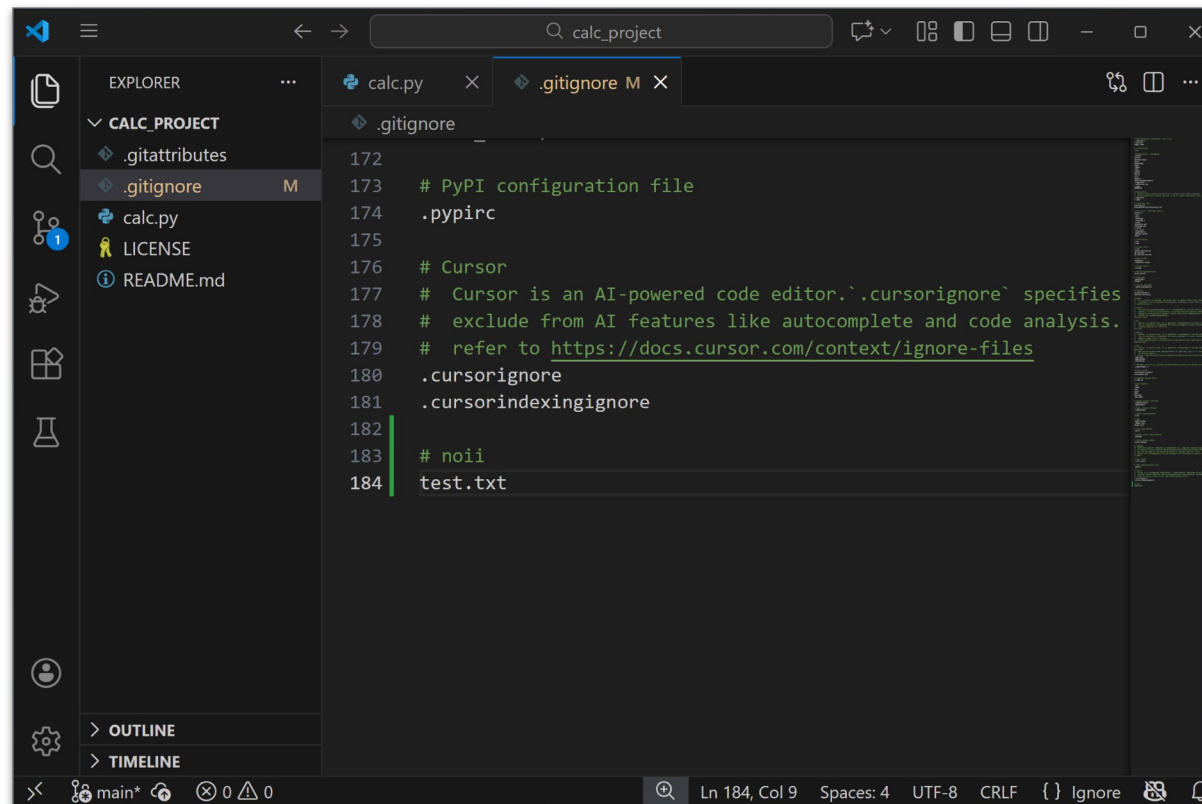
"이 작업을 통째로 취소해도 아깝지 않은 단위"

추적 불필요

프로젝트 폴더 내에서 다른 사람과 공유할 필요가 없거나
임시로 생성된 파일들이 있을 수 있습니다
이런 파일들은 기록을 남길 필요가 없어요

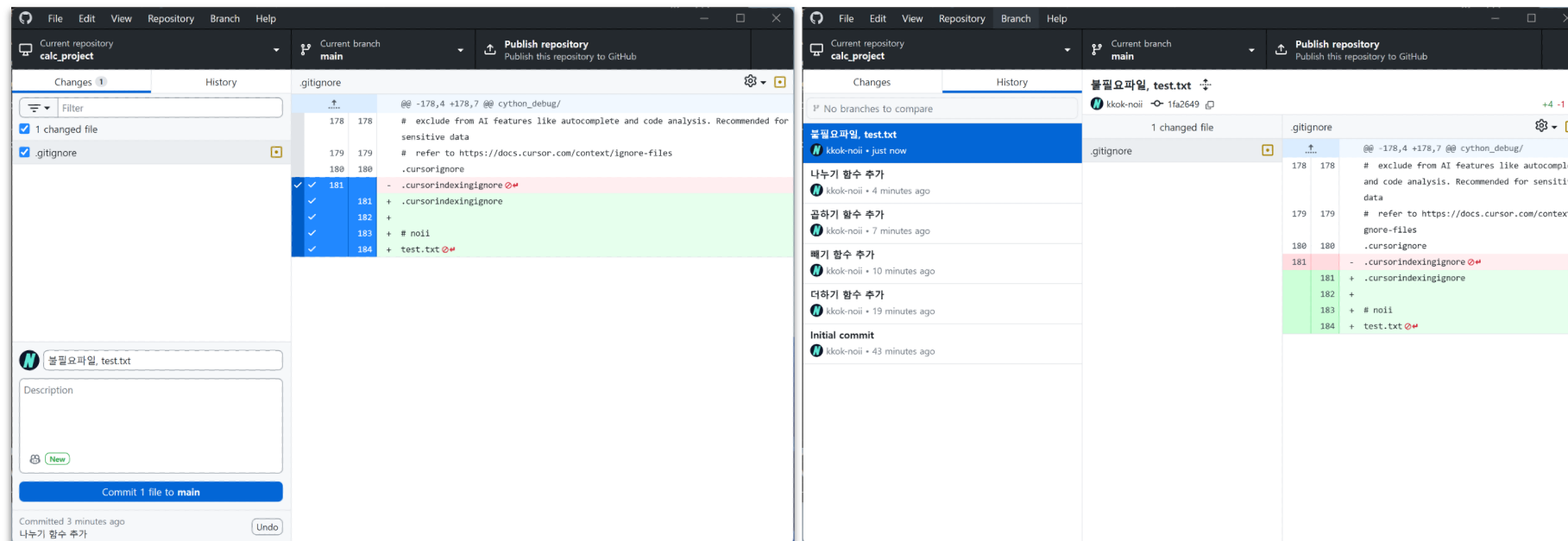
text.txt을 .gitignore 파일에 추가해보세요

특정 파일명, 폴더명, 확장자를 추가할 수 있어요



추적 불필요

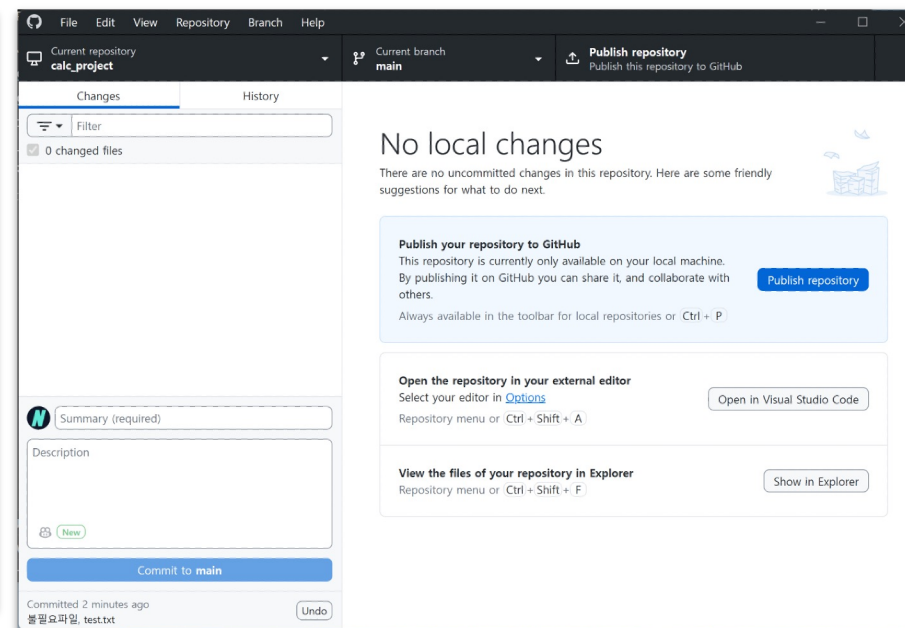
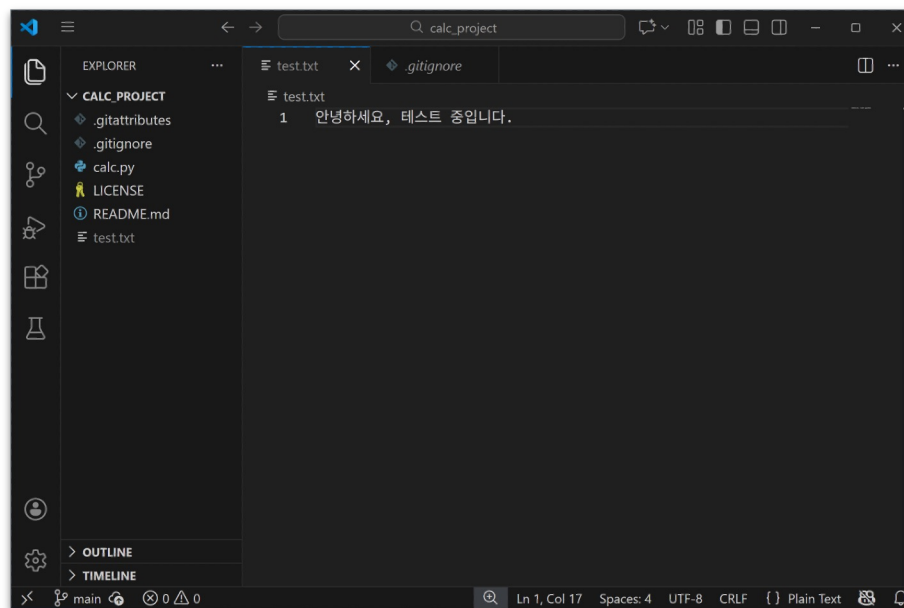
.gitignore 또한 Repository 내에서 추적하는 파일로 변경 이력을 추적합니다



추적 불필요

test.txt 파일을 생성하고 내용을 작성해보면 ...

test.txt의 파일명을 바꿔보세요



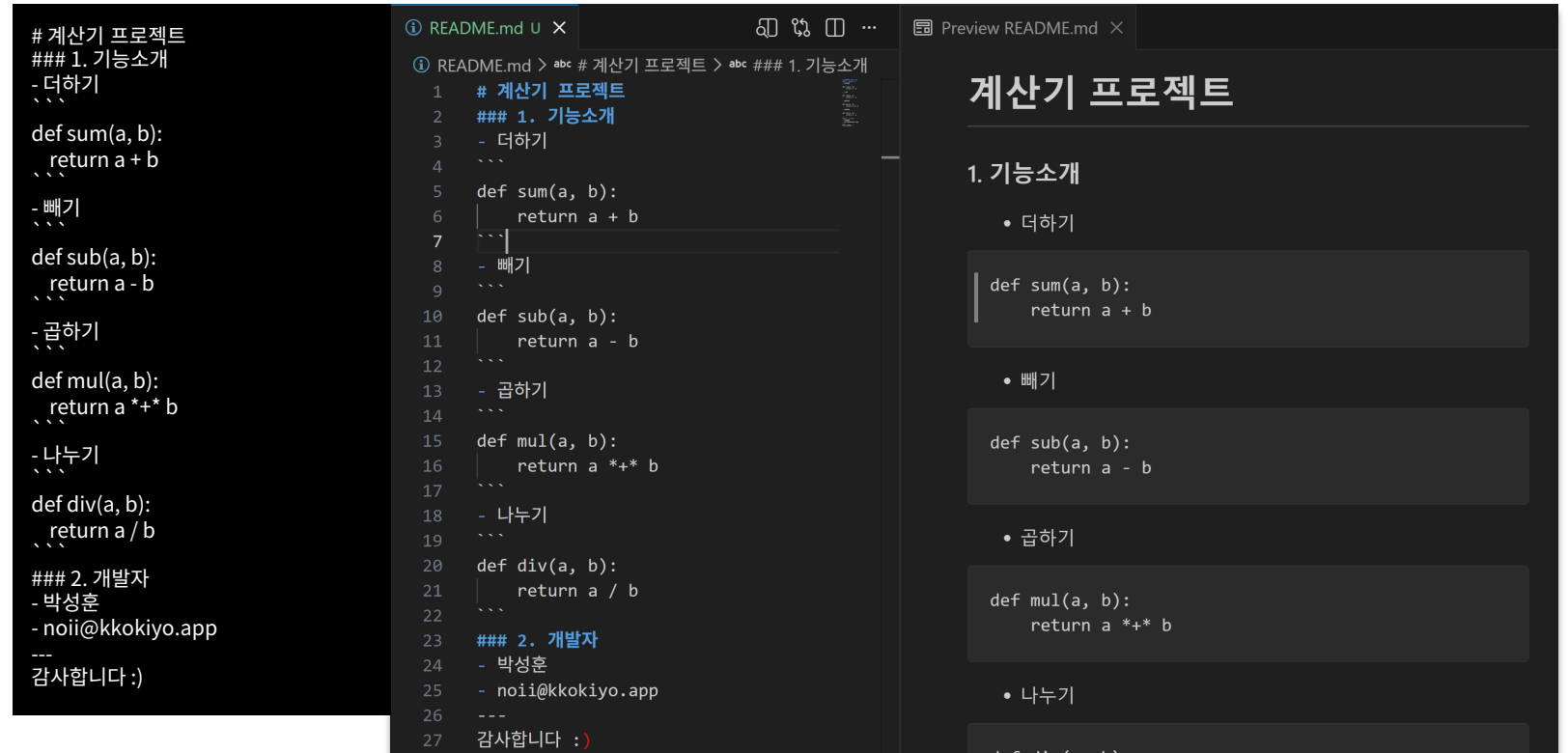
SESSION 5

Github 배포하기

Github 배포하기

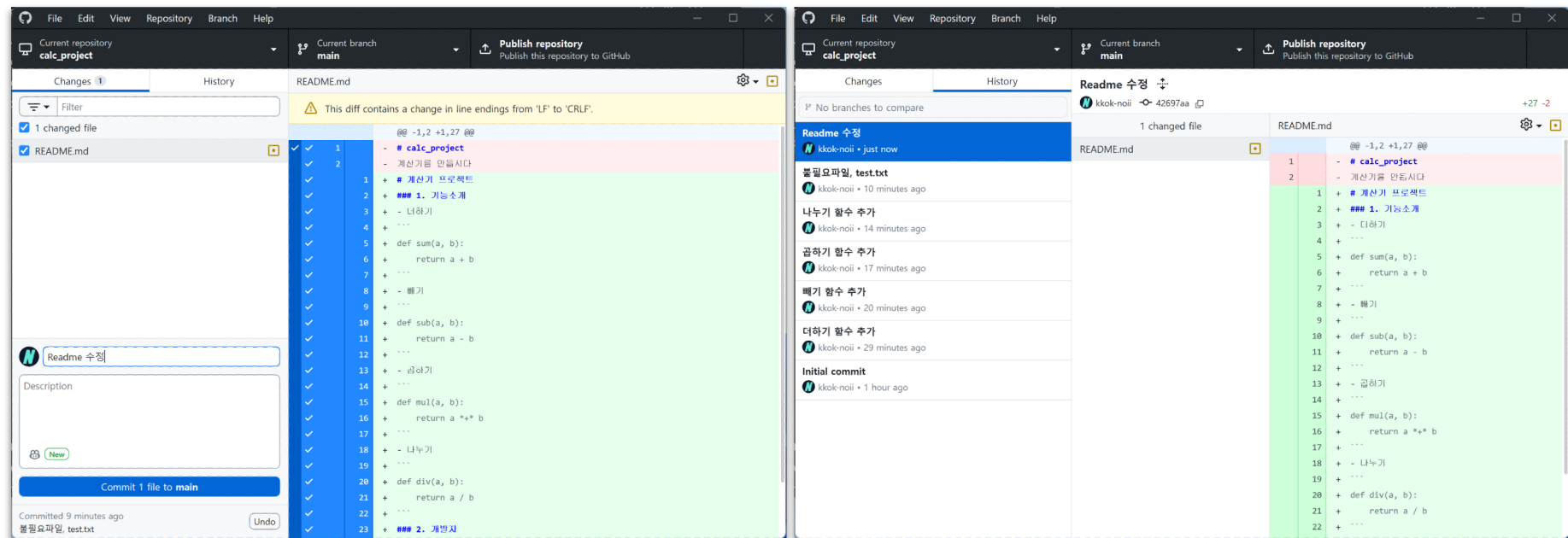
README 파일 작성하기

Markdown 방식으로 작성



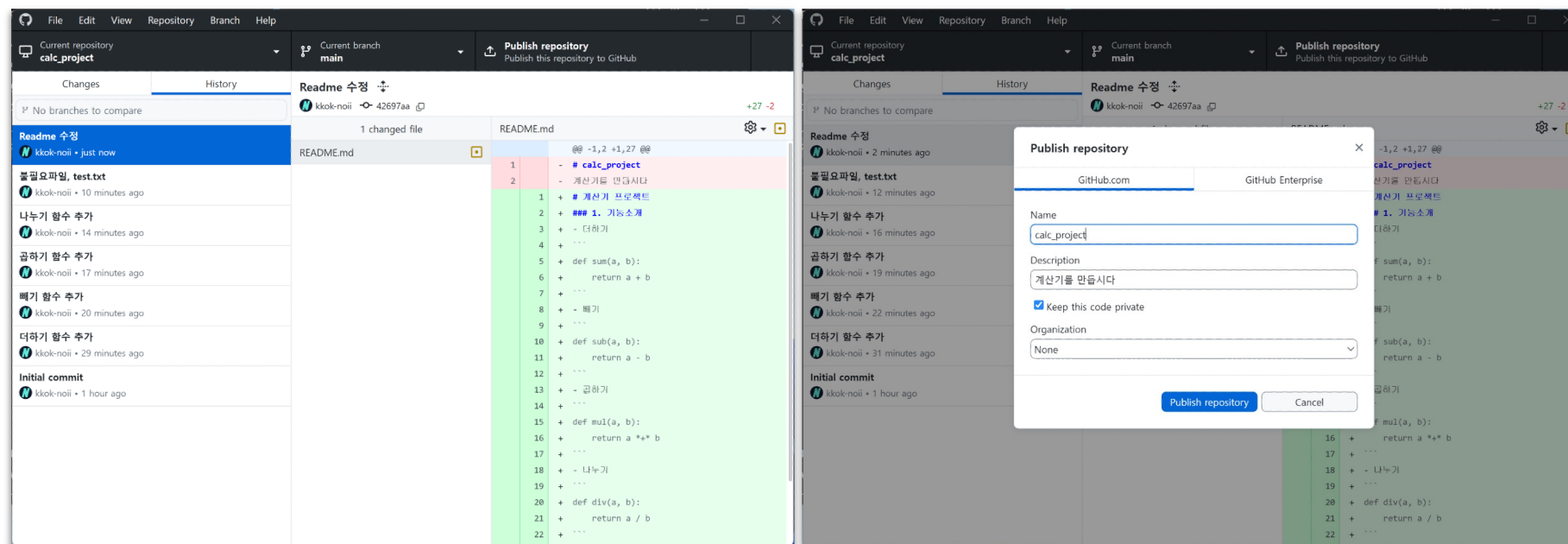
Github 배포하기

README 파일 수정 내용도 Commit 해줍니다



Github 배포하기

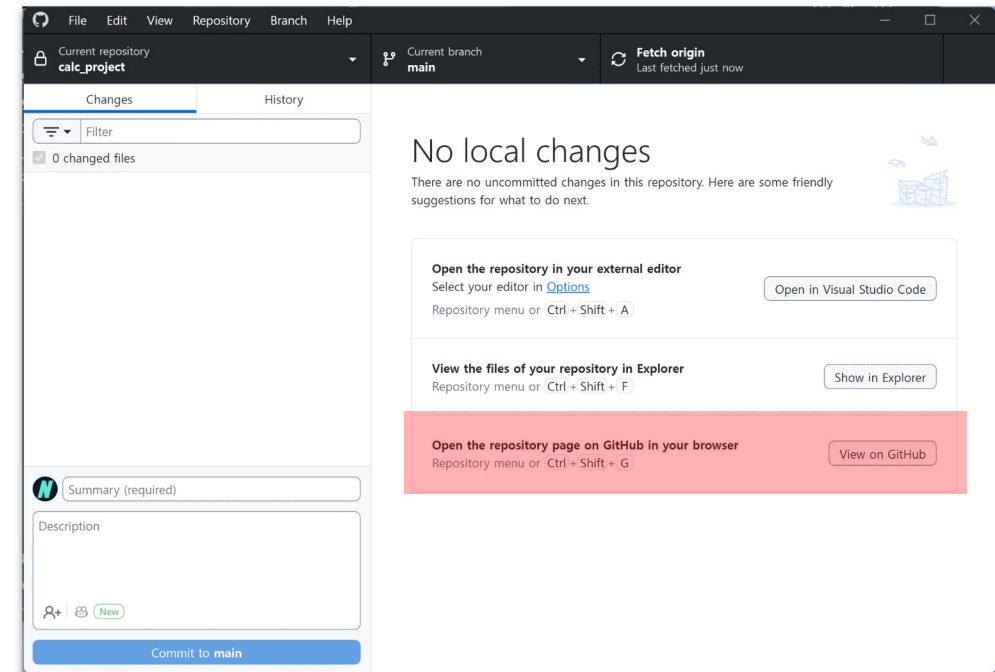
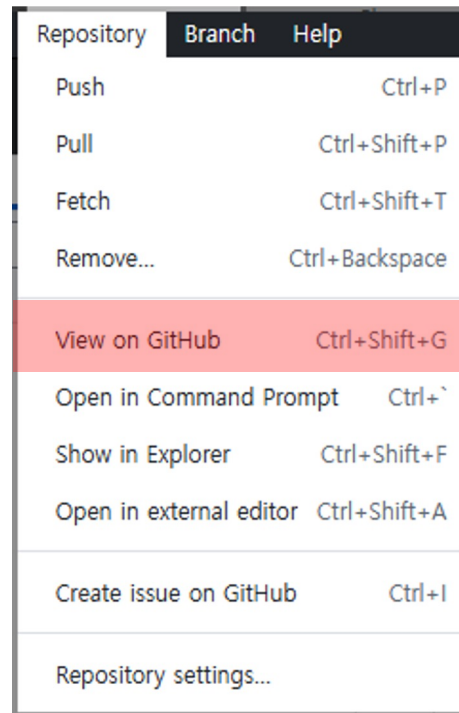
Github desktop 상단의 **Publish repository**
Keep this code private 체크 시 비공개로 배포



Github 배포하기

Github에 배포가 완료됐어요

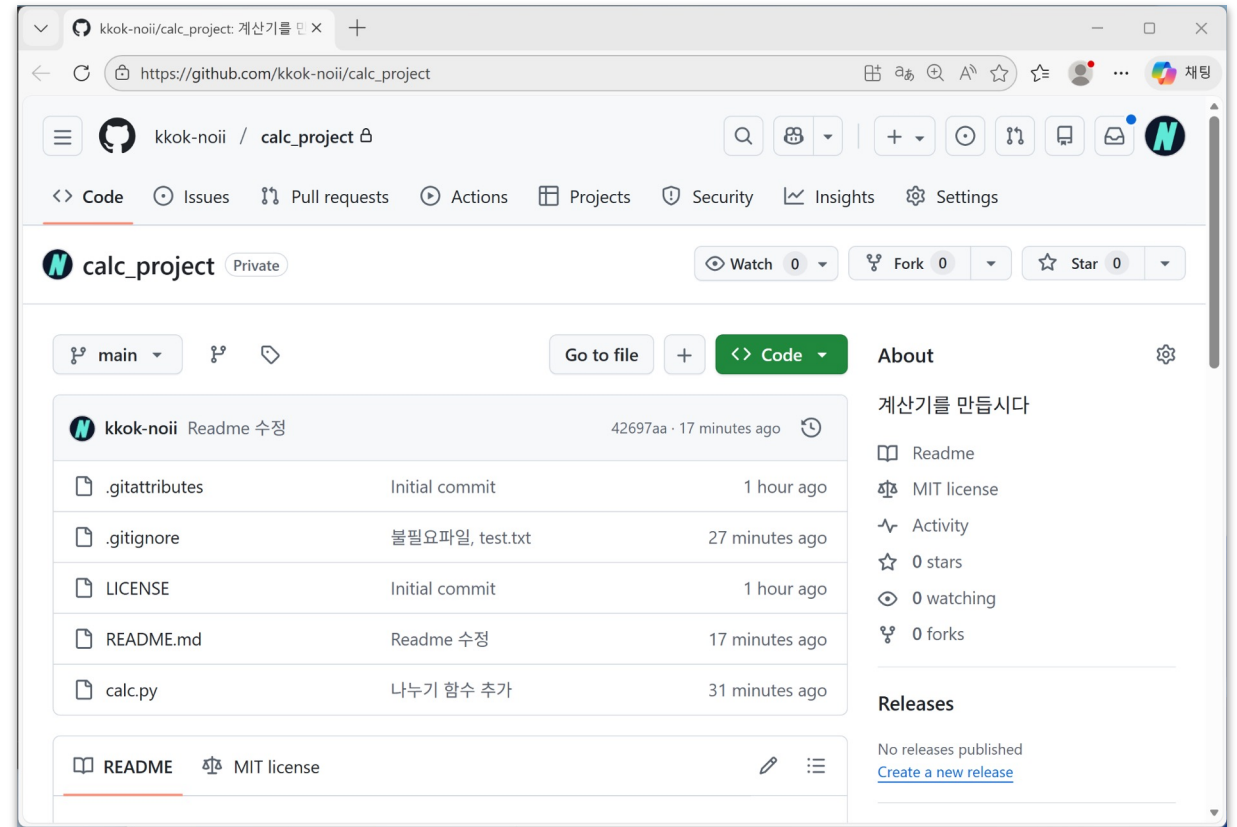
Github page에서 확인해봅시다



Github 배포하기

Github에 배포가 완료됐어요

Github page에서 확인해봅시다



SESSION 6

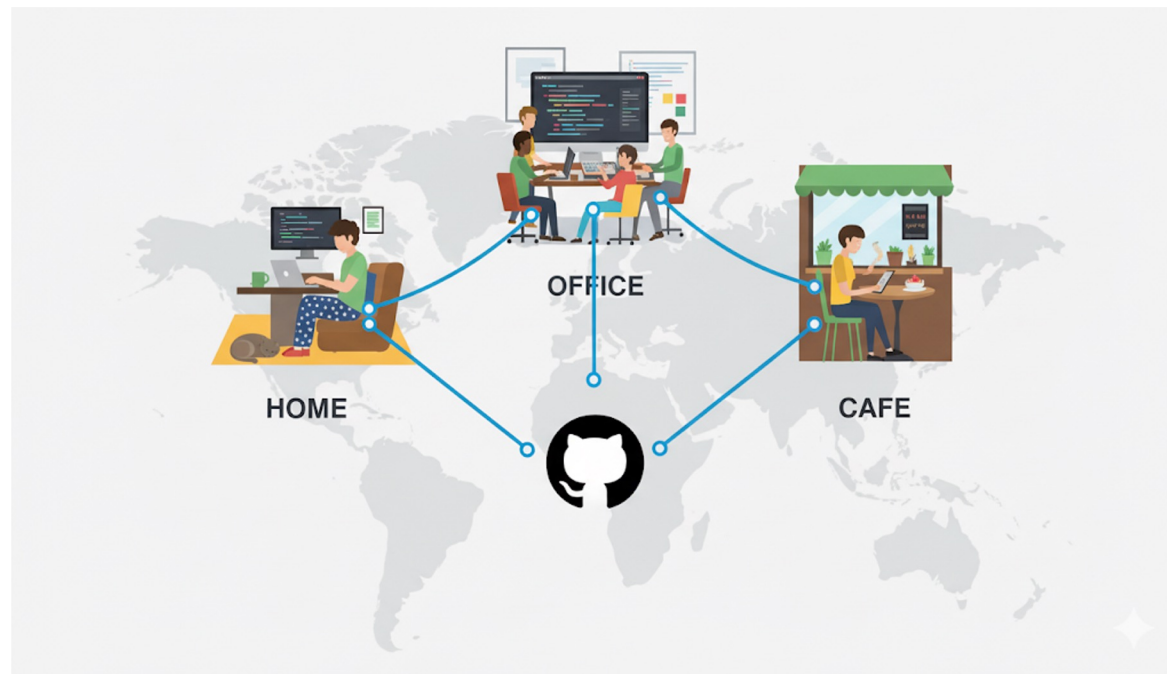
Push, Pull, 그리고 충돌

다른 공간에서 작업하기

서로 다른 공간에서 하나의 프로젝트를 작업해봅시다

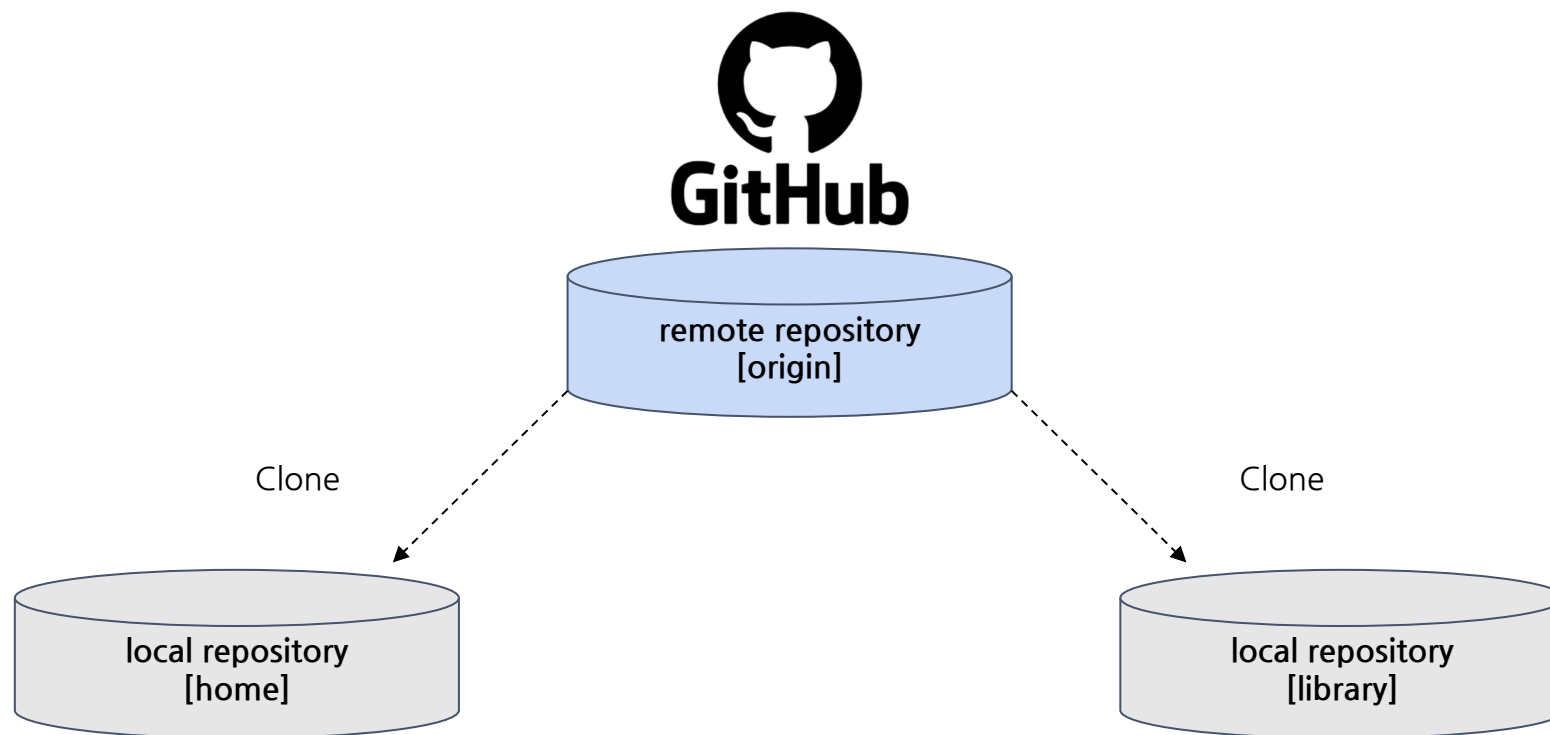
사무실에서 - 집에서, 또는 A - B 두 사람이

USB나 외장하드에 소스코드를 담아 작업할 수 있지만, Github를 활용하면 더 좋아요



다른 공간에서 작업하기

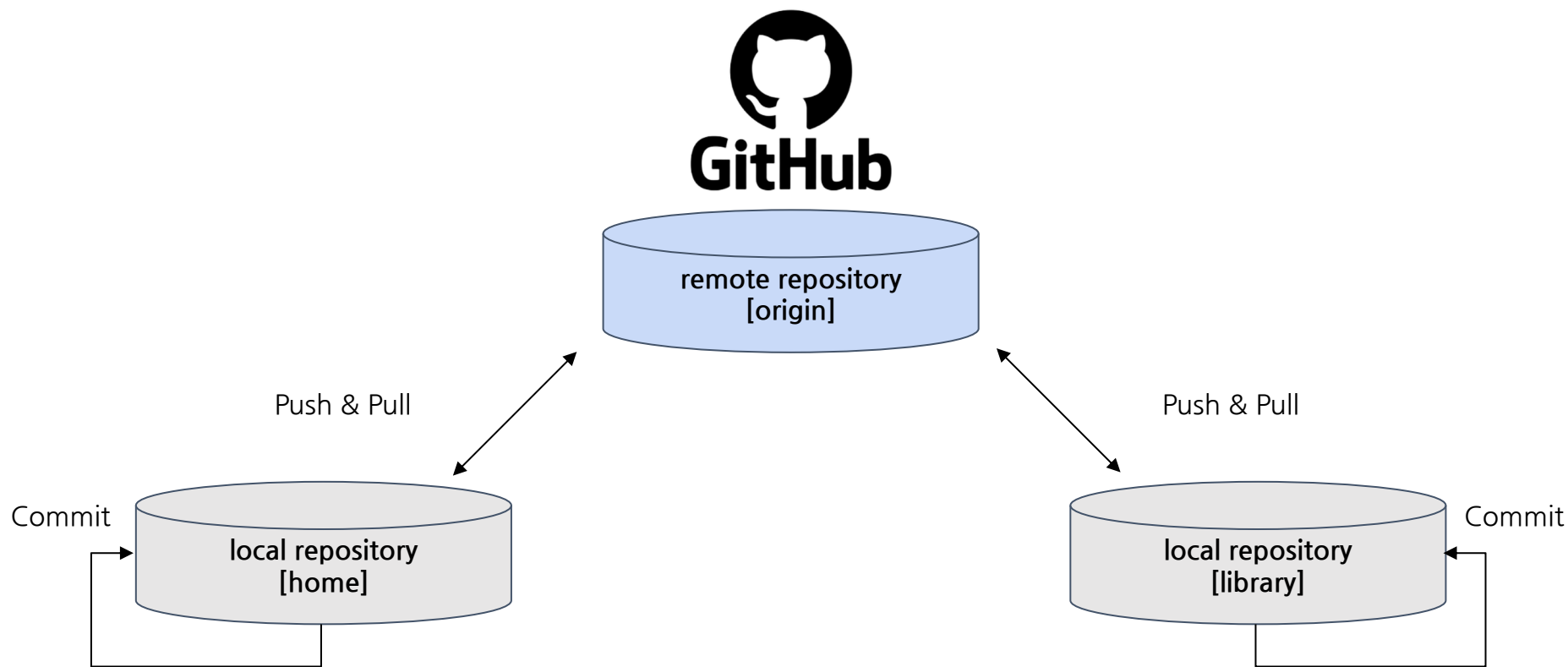
Remote repository를 원본 코드로하면
어디서든 해당 코드를 가져올 수 있어요



공유하기

다른 공간에서 작업하기

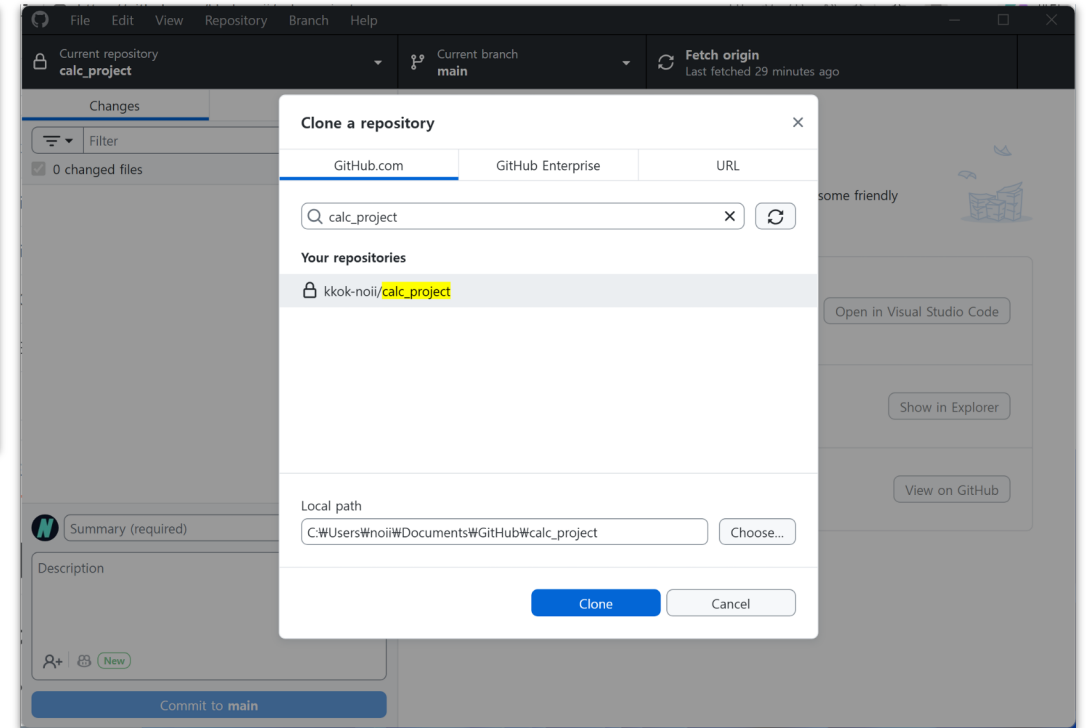
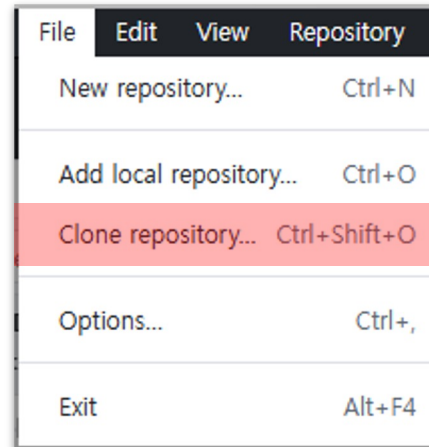
각각의 작업한 소스코드는 **Push & Pull**을 통해 공유할 수 있어요



Github 코드 가져오기

Clone을 통하여 기존 Repository 가져오기

File > Clone repository

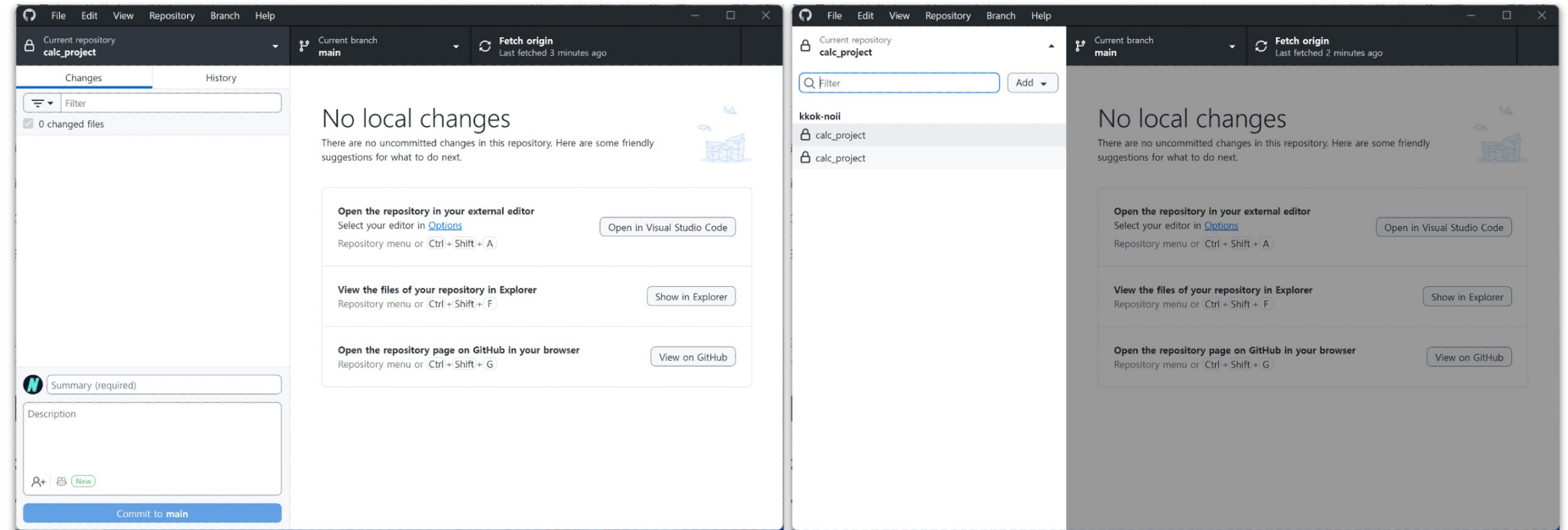


Github 코드 가져오기

두개의 Repository 확인 가능

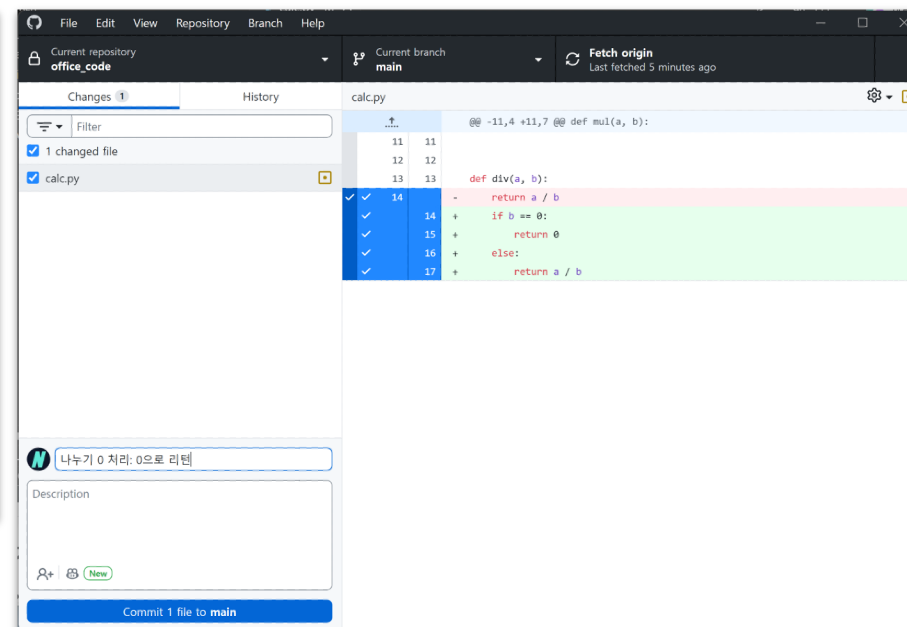
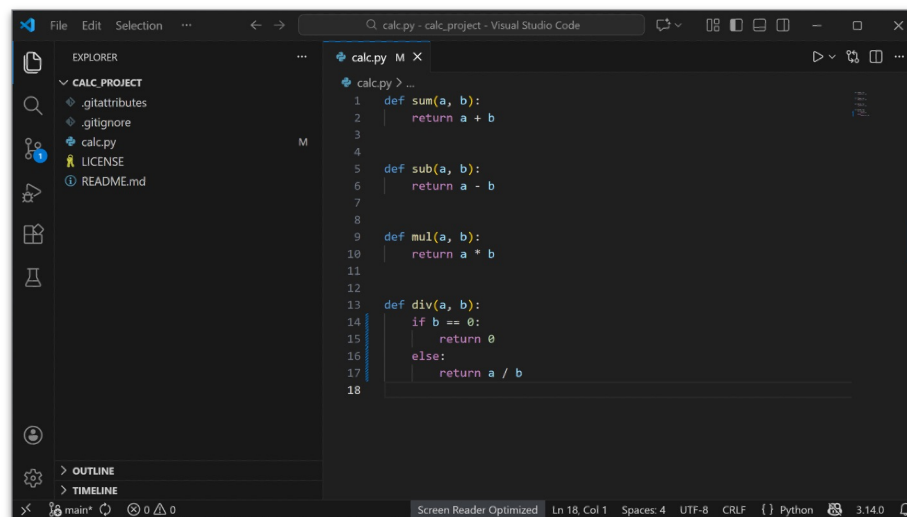
이름이 동일하여 혼당되니 별명을 지어 구분해줍니다

(마우스 우클릭 > Create alias)



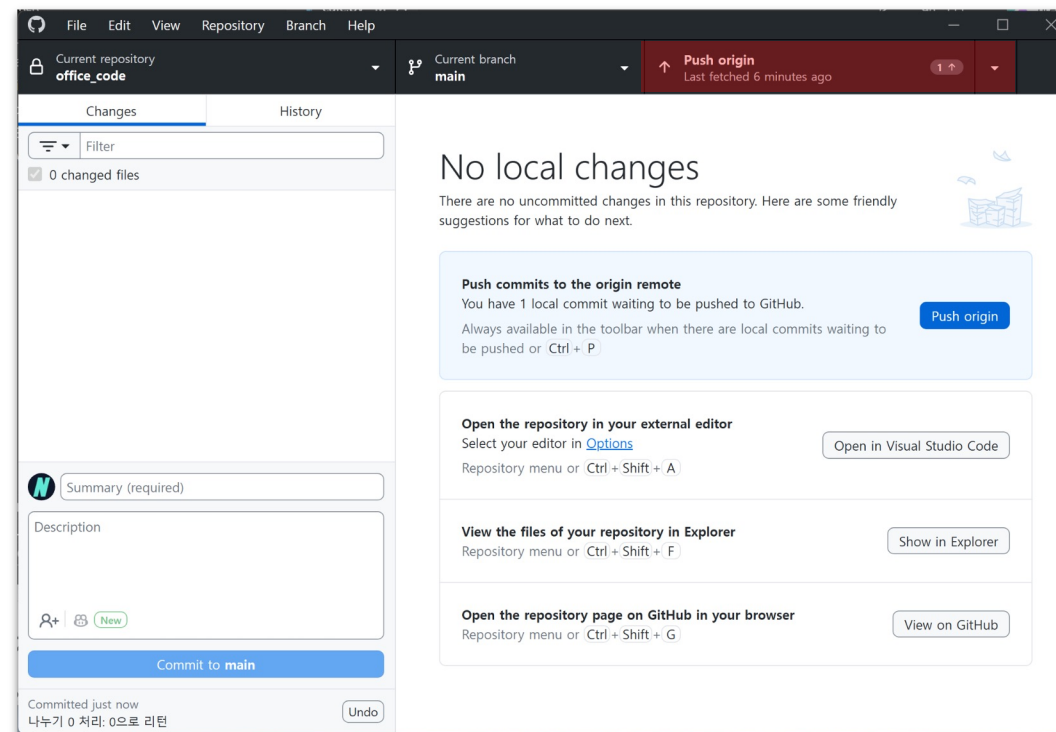
코드 공유하기

[Office] 코드를 수정하여 Commit



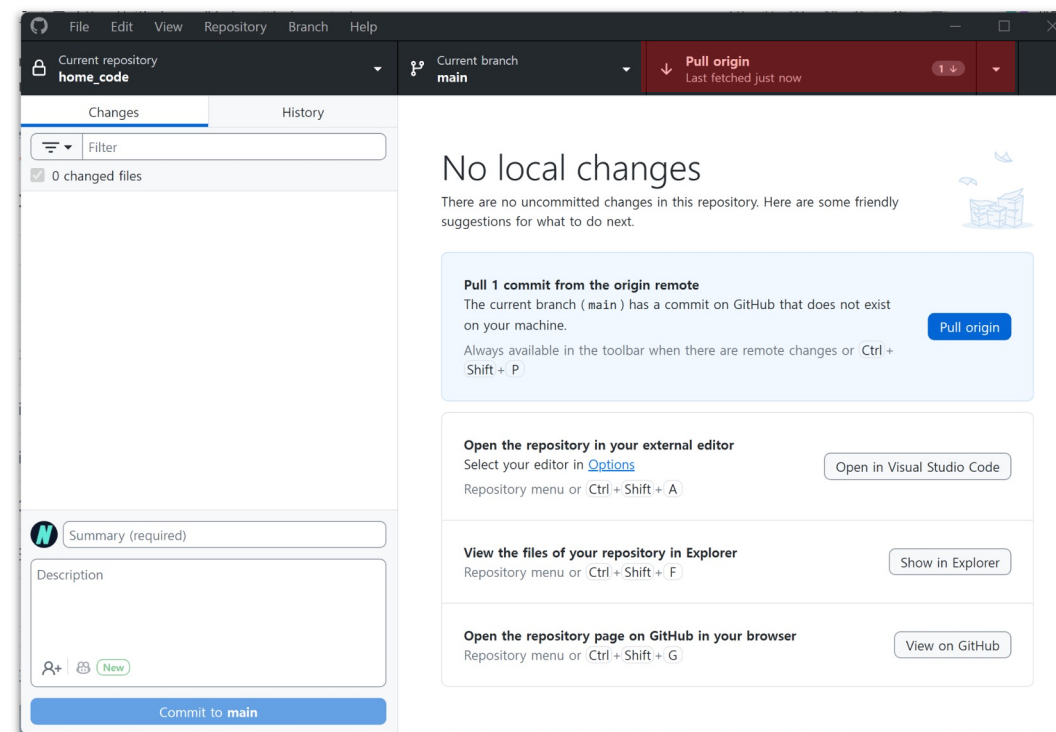
코드 공유하기

[Office] push origin을 통해서 Github에 변경 내역 공유하기



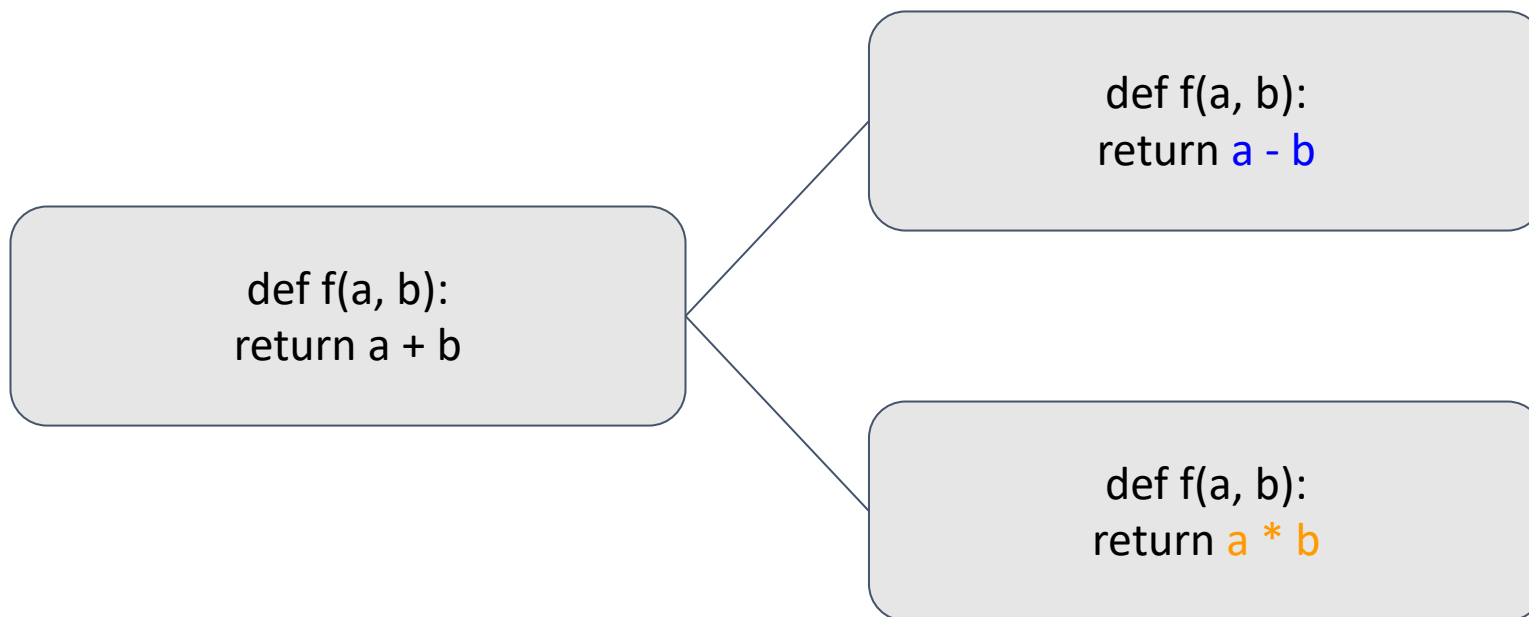
코드 공유하기

[Home] pull origin을 통해서 Github에서 최신 코드 내려받기



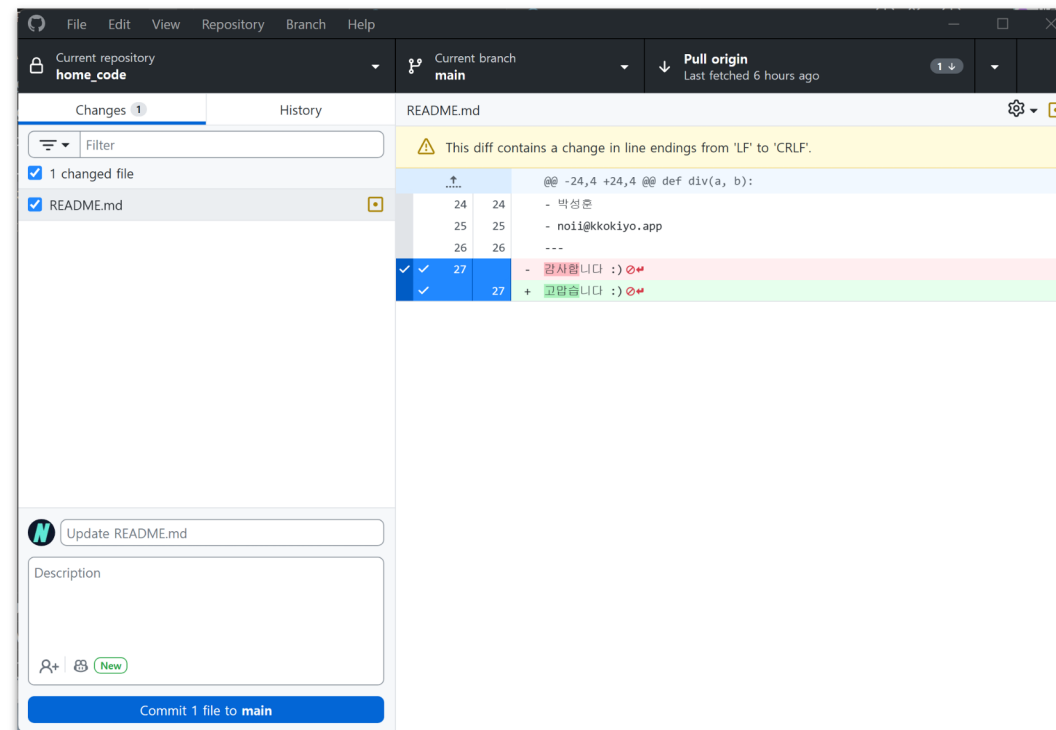
충돌 해결하기

동일한 코드를 동 시점에 두가지 버전으로 수정을 하면 충돌이 발생합니다
그 상황을 인지하고, 개발자가 직접 해결해줘야 합니다



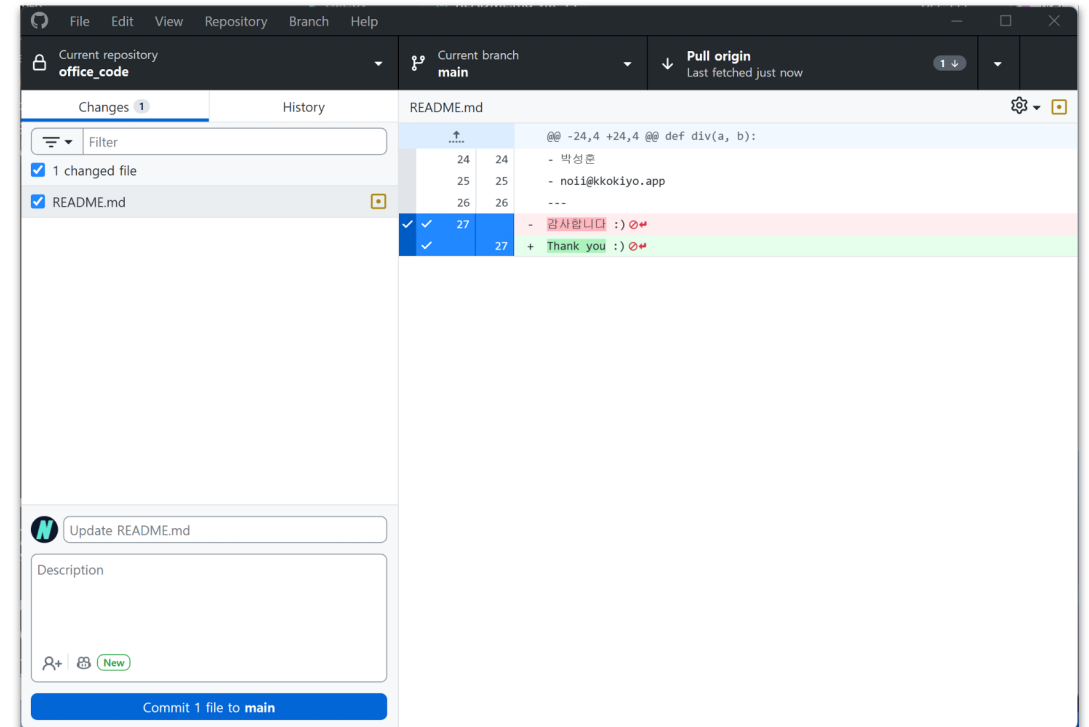
충돌 해결하기

[Home] README.md의 마지막, **감사합니다:)**를 **고맙습니다:)**로 수정
Commit, Push까지 진행해주세요



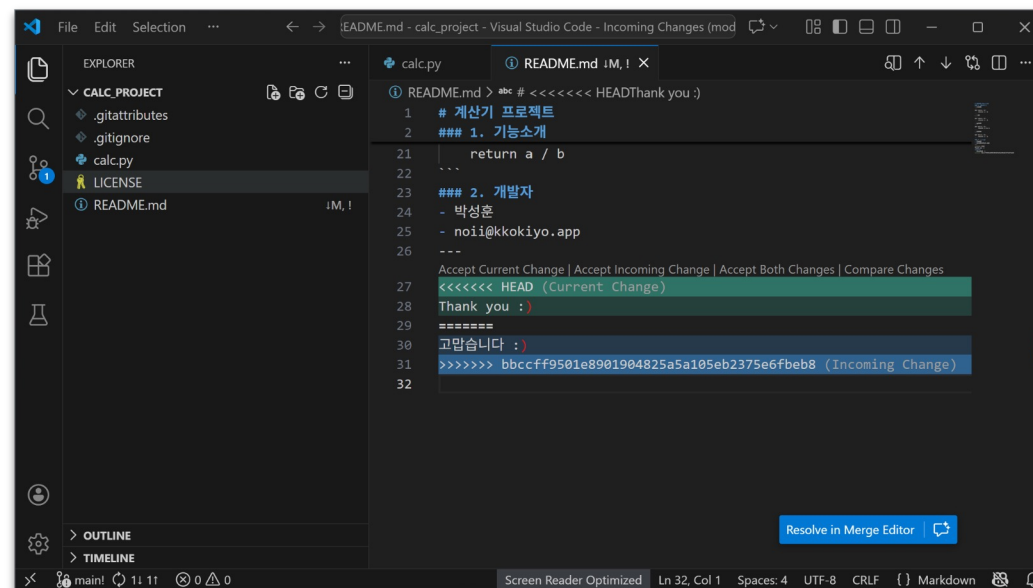
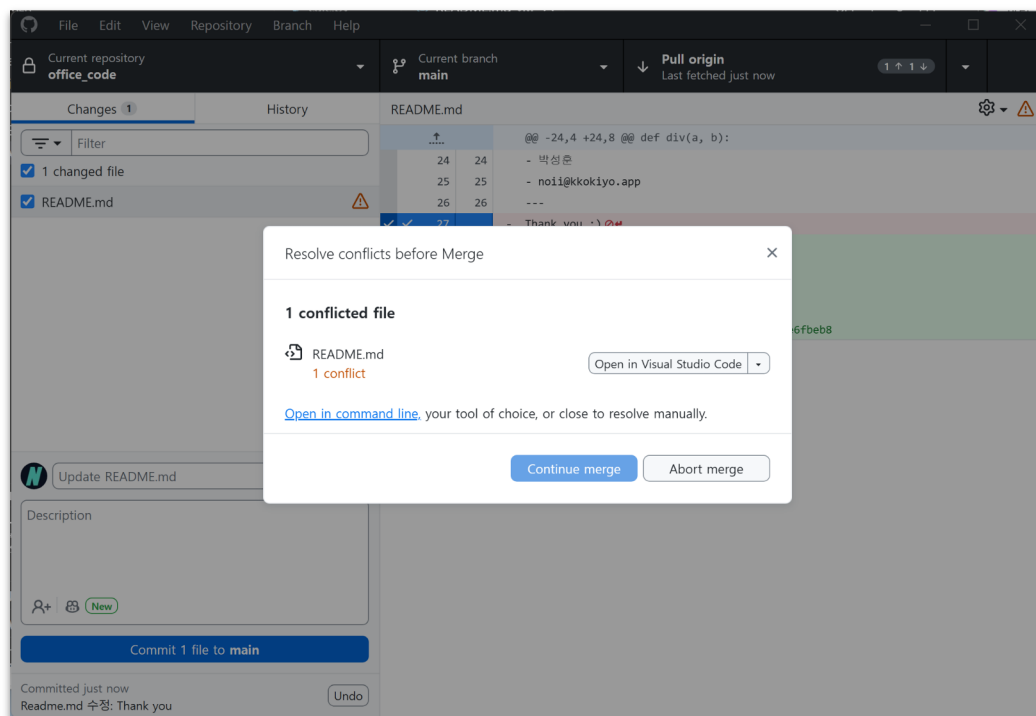
충돌 해결하기

[Office] Pull을 하지 않은 상태에서
README.md의 마지막, **감사합니다:)**를 **Thank you:)**로 수정
Commit, Push를 하려면 우선 Pull이 필요하다고...



충돌 해결하기

[Office] Pull을 받으면 충돌 발생



충돌 해결하기

아래처럼 텍스트로 충돌 부분을 표기합니다

<<<<<< HEAD

[내가 작성한 부분]

=====

[충돌난 다른 부분]

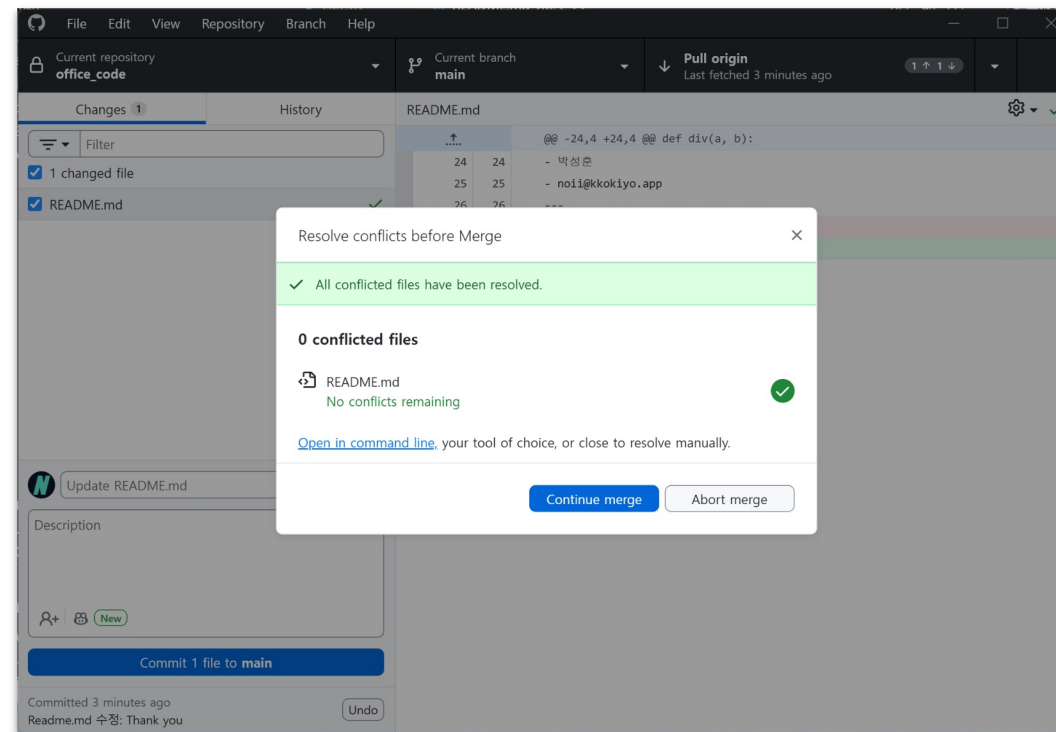
>>>>>> **bbccff9501e8901904825a5a105eb2375e6fbeb8**

충돌 해결하기

충돌난 부분에 대해서 의사 결정

내가 맞는지, 이전이 맞는지, 둘 다 맞는지...

이후 Merge가 가능합니다



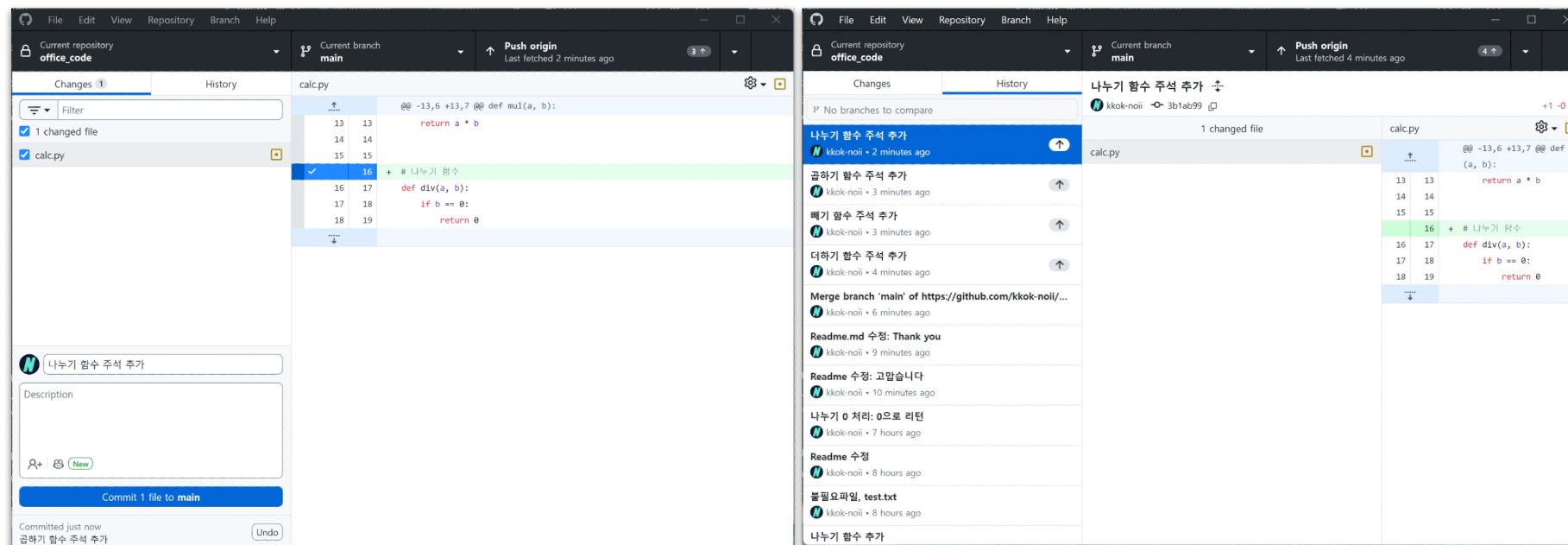
SESSION 8

커밋 합치기

커밋 합치기

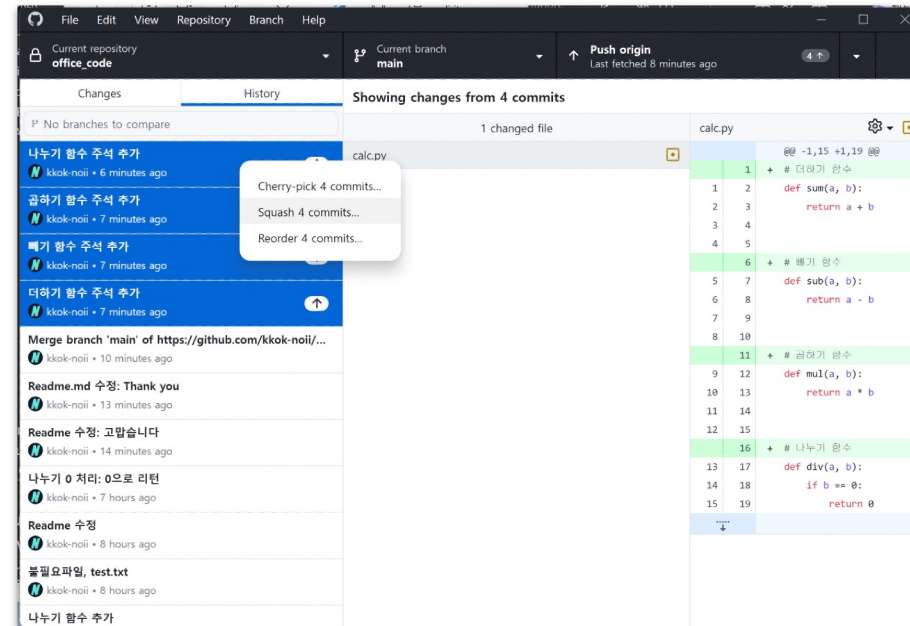
각 함수에 주석을 추가해줬습니다

하나의 주석을 추가해줄 때 마다, Commit을 하니 4개의 Commit이 생겼습니다



커밋 합치기

기록을 세세히 남기는것도 좋지만, 불필요하게 디테일한듯 합니다
합치고자 하는 Commit들을 선택하고 Squash



커바 합치기

하나의 대표 Commit으로 남깁니다

각 함수 주석 작성

